

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ



دانشگاه صنعتی اصفهان

دانشکده مهندسی برق و کامپیوتر

**توسعه پلتفرم خرید و فروش توکن های غیر قابل
معاوضه (NFT) مبتنی بر بلاکچین
محمدرضا ماجد**

استاد راهنما

دکتر فانیان

شهریور ۱۴۰۴

فهرست مطالب

عنوان	صفحه
فهرست مطالب	چهار
فهرست تصاویر	هفت
چکیده	۱
فصل اول: مقدمه	۲
۱-۱ تعریف مسئله و اهمیت موضوع	۲
۲-۱ اهداف پروژه	۳
۳-۱ ساختار گزارش	۳
فصل دوم: مبانی نظری	۵
۱-۲ بلاکچین	۵
۲-۲ قرارداد هوشمند	۶
۳-۲ اتریوم	۶
۱-۳-۲ Hardhat	۷
۴-۲ توکن های غیر قابل معاوضه	۷
۱-۴-۲ استاندارد ERC-721	۷
۵-۲ ذخیره سازی غیر متمرکز توکن غیر قابل معاوضه	۸
۱-۵-۲ پروتکل IPFS	۸
۶-۲ کیف پول های دیجیتال و مدیریت هویت در Web3	۹
فصل سوم: نیازمندی ها	۱۰
۱-۳ نیازمندی های کاربردی	۱۰

۱۱	۱-۱-۳ مدیریت کاربر و احراز هویت
۱۱	۲-۱-۳ چرخه حیات توکن
۱۱	۳-۱-۳ عملیات مالی
۱۲	۴-۱-۳ تعاملات کاربری و بازار
۱۲	۵-۱-۳ مدیریت قرارداد و امنیت
۱۳	۲-۳ نیازمندی‌های غیر کاربردی
۱۳	۱-۲-۳ امنیت
۱۳	۲-۲-۳ اطمینان و استحکام
۱۳	۳-۲-۳ تجربه کاربری و سازگاری
۱۴	فصل چهارم: معماری و طراحی سیستم
۱۴	۱-۴ معماری سطح بالا
۱۶	۲-۴ فناوری‌های استفاده شده
۱۷	۳-۴ طراحی اجزای سیستم
۱۸	۱-۳-۴ طراحی قرارداد هوشمند
۲۰	۲-۳-۴ طراحی بک‌اند
۲۱	۳-۳-۴ طراحی فرانت‌اند
۲۲	۴-۴ چرخه حیات توکن
۲۳	فصل پنجم: پیاده‌سازی
۲۳	۱-۵ پیاده‌سازی قرارداد هوشمند
۲۳	۱-۱-۵ ساختار اصلی و متغیرهای قرارداد
۲۴	۲-۱-۵ توابع اصلی قرارداد
۲۵	۲-۵ پیاده‌سازی بک‌اند
۲۶	۲-۲-۵ پیاده‌سازی API
۲۷	۳-۵ پیاده‌سازی فرانت‌اند
۲۸	۲-۳-۵ مدیریت وضعیت و تعامل با بلاکچین
	۴-۵ پیاده‌سازی احراز هویت

۳۱	 ۱-۴-۵ پیاده سازی سمت بک اند (NextAuth.js)
۳۲	 ۲-۴-۵ پیاده سازی سمت فرانت اند
۳۴	 فصل ششم: اجرا و تست
۳۴	 ۱-۶ راهنمای اجرای پروژه
۳۴	 ۱-۱-۶ پیش نیازها و نصب وابستگی ها
۳۵	 ۲-۱-۶ پیکربندی متغیرهای محیطی
۳۵	 ۳-۱-۶ راه اندازی و مدیریت پایگاه داده
۳۵	 ۴-۱-۶ اجرای شبکه بلاکچین محلی
۳۶	 ۵-۱-۶ کامپایل و استقرار قرارداد هوشمند
۳۷	 ۶-۱-۶ انتقال ABI
۳۷	 ۷-۱-۶ اجرای سرور توسعه
۳۷	 ۸-۱-۶ پیکربندی کیف پول MetaMask برای تست
۳۹	 ۲-۶ سناریوهای تست
۳۹	 ۱-۲-۶ تست ورود و امضای دیجیتال
۴۰	 ۲-۲-۶ تست ساخت و لیست کردن توکن
۴۱	 ۳-۲-۶ تست خرید یک توکن
۴۲	 ۴-۲-۶ تست لغو فروش
۴۲	 ۵-۲-۶ تست فروش مجدد
۴۳	 ۶-۲-۶ تست لیست علاقه مندی ها و گزارش تخلف
۴۴	 ۷-۲-۶ تست برداشت وجه
۴۵	 ۸-۲-۶ تست قابلیت توقف اضطراری قرارداد
۴۷	 فصل هفتم: نتیجه گیری و کارهای آینده
۴۷	 ۱-۷ نتیجه گیری
۴۸	 ۲-۷ کارهای آینده
۵۰	 مراجع

فهرست تصاویر

شکل ۱-۱: نمودار فروش توکن غیرقابل معاوضه ^۱ در بازه ۲۰۲۱ تا ۲۰۲۵.....	۳
شکل ۱-۲: روند کلی پیاده‌سازی و اجرای قراردادهای هوشمند در شبکه اتریوم.....	۷
شکل ۲-۲: نمودار ارتباطات بین اجزای مختلف در متد IPFS.....	۸
شکل ۱-۳: نمودار موارد استفاده پروژه.....	۱۲
شکل ۱-۴: نمودار ارتباطات و پیام‌ها در Prisma.....	۱۷
شکل ۲-۴: نمودار ماشین حالت چرخه حیات توکن.....	۲۲
شکل ۱-۵: نمودار ارتباط بین موجودیت‌های پایگاه داده.....	۲۶
شکل ۲-۵: نمونه کد استفاده از تابع قرارداد در فرانت‌اند.....	۲۹
شکل ۳-۵: نمودار فعالیت برای فرآیند ایجاد توکن.....	۲۹
شکل ۴-۵: نمودار توالی فرآیند خرید توکن.....	۳۰
شکل ۵-۵: کد تابع fetchMyNFTs در فایل NFTMarketplaceContext.....	۳۰
شکل ۶-۵: کد تابع authorize در API احراز هویت.....	۳۱
شکل ۷-۵: نمودار توالی برای فرآیند ورود و احراز هویت.....	۳۲
شکل ۱-۶: خروجی دستور اجرای hardhat.....	۳۶
شکل ۲-۶: خروجی دستور پیاده‌سازی قرارداد.....	۳۶
شکل ۳-۶: صفحه تنظیمات شبکه محلی در metamask.....	۳۸
شکل ۴-۶: صفحه ورود و امضا در metamask.....	۳۹
شکل ۵-۶: صفحه ساخت توکن و نتیجه در پایگاه داده.....	۴۰
شکل ۶-۶: صفحه خرید توکن و نتیجه در جدول معاملات پایگاه داده و پروفایل.....	۴۱
شکل ۷-۶: صفحه کنسل کردن فروش.....	۴۲
شکل ۸-۶: صفحه فروش مجدد توکن و اجازه برداشت توکن.....	۴۳
شکل ۹-۶: صفحه برداشت موجودی و تغییرات کیف پول.....	۴۴
شکل ۱۰-۶: نمونه خروجی توابع حساس قرارداد در حالت.....	۴۶

چکیده

با گسترش روزافزون فناوری بلاکچین^۱، کاربردهای آن از ارزهای دیجیتال^۲ فراتر رفته و به حوزه‌های جدیدی مانند اثبات مالکیت دارایی‌های دیجیتال گسترش یافته است. توکن‌های غیرقابل معاوضه^۳ به عنوان یکی از مهم‌ترین بخش‌های این فناوری، راهکاری مناسب برای ثبت و انتقال مالکیت منحصر به فرد دارایی‌های دیجیتال ارائه می‌دهند. هدف اصلی این پروژه، طراحی و پیاده‌سازی یک بازار آنلاین برای ایجاد، خرید و فروش این توکن‌ها است.

معماری این پروژه بر سه بخش اصلی استوار است: یک قرارداد هوشمند^۴ مبتنی بر Solidity برای مدیریت منطق توکن‌ها بر روی بلاکچین، یک بک‌اند^۵ توسعه‌یافته با Next.js برای مدیریت داده‌های خارج از زنجیره و یک فرانت‌اند^۶ پویا برای تعامل با کاربر. برای ورود امن و یکپارچه کاربران به اکوسیستم Web3، از مکانیزم احراز هویت مبتنی بر کیف پول دیجیتال^۷ استفاده شده است. همچنین، به منظور غیرمتمرکز کردن تا جای ممکن، داده‌های اصلی توکن‌ها بر روی IPFS^۸ ذخیره می‌شوند.

نتیجه نهایی، یک پلتفرم جامع و غیرمتمرکز است که به کاربران امکان می‌دهد با استفاده از کیف پول MetaMask خود، به سادگی وارد سیستم شده، توکن‌های جدید ایجاد کنند و در یک بازار امن به خرید و فروش آن‌ها بپردازند.

واژه‌های کلیدی: ۱- بلاکچین، ۲- اتریوم^۹، ۳- قرارداد هوشمند، ۴- توکن غیرقابل معاوضه، ۵- فرانت‌اند، ۶- بک‌اند

¹ Blockchain

² Crypto currency

³ NFT (non-fungible token)

⁴ SmartContract

⁵ Backend

⁶ Frontend

⁷ Crypto wallet

⁸ InterPlanetary File System

⁹ Ethereum

فصل اول

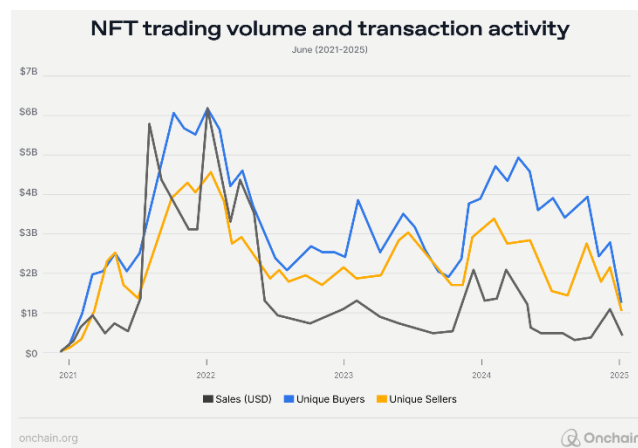
مقدمه

۱-۱ تعریف مسئله و اهمیت موضوع

در دهه اخیر، فناوری بلاکچین تحولی بنیادین در نحوه ذخیره‌سازی و انتقال داده‌ها به صورت امن و غیرمتمرکز ایجاد کرده است. این فناوری که در ابتدا با ارزهای دیجیتال مانند بیت‌کوین^۱ شناخته می‌شد، به سرعت پتانسیل خود را در حوزه‌های گسترده‌تری به نمایش گذاشت. یکی از جدیدترین کاربردهای بلاکچین، ایجاد و مدیریت دارایی‌های دیجیتال منحصربه‌فرد از طریق NFT است. رشد بازار این توکن‌ها در سال‌های اخیر بسیار چشمگیر بوده و فرصت‌های جدیدی برای هنرمندان، تولیدکنندگان محتوا و مجموعه‌داران ایجاد کرده است.

کاربران در دنیای NFT همچنان با چالش‌هایی مواجه هستند. بسیاری از پلتفرم‌های موجود یا دارای رابط‌های کاربری پیچیده هستند یا کارمزد بالایی برای تراکنش‌ها دریافت می‌کنند. همچنین، کمبود بسترهای امن و کاربرپسند که فرآیند ایجاد، خرید و فروش را برای کاربران عادی تسهیل کنند، یک نیاز اساسی در این حوزه محسوب می‌شود. این پروژه در پاسخ به همین نیاز تعریف شده است و قصد دارد یک پلتفرم جامع و در عین حال ساده را برای مدیریت توکن‌های غیرقابل معاوضه ارائه دهد که در آن کاربران به راحتی قادر به تعامل با این دنیای جدید باشند.

¹ Bitcoin



شکل ۱-۱: نمودار فروش NFT در بازه ۲۰۲۱ تا ۲۰۲۵

۱-۲ اهداف پروژه

برای پاسخ به چالش‌های مطرح‌شده، اهداف مشخصی برای این پروژه تعریف شده است که به دو دسته اصلی و جزئی تقسیم می‌شوند.

هدف اصلی: هدف اصلی این پروژه، طراحی و پیاده‌سازی یک بازار آنلاین غیرمتمرکز برای توکن‌های غیرقابل معاوضه است که تمامی مراحل از ورود کاربر تا تبادل دارایی را پوشش دهد.

اهداف جزئی: برای دستیابی به هدف اصلی، اهداف جزئی زیر دنبال می‌شوند:

- **پیاده‌سازی قابلیت‌های اصلی بازار:** ایجاد امکان ساخت، لیست کردن برای فروش، و خرید توکن‌های غیرقابل معاوضه توسط کاربران.
- **ایجاد یک پلتفرم امن:** طراحی و پیاده‌سازی سیستم احراز هویت امن و مدرن مبتنی بر کیف پول دیجیتال (SIWE)^۱، که نیاز به نام کاربری و رمز عبور را حذف کرده و تنها برپایه کیف پول دیجیتال استوار است.
- **فراهم کردن رابط کاربری ساده:** توسعه یک رابط کاربری ساده، مدرن و جذاب که تجربه کاربری مثبتی را برای افراد، حتی کسانی که با فناوری بلاکچین آشنایی کمی دارند، فراهم آورد.
- **تضمین عدم تمرکز:** استفاده از IPFS برای ذخیره‌سازی داده‌های اصلی توکن‌ها به منظور جلوگیری از وابستگی به سرور و افزایش پایداری پلتفرم.

۱-۳ ساختار گزارش

- **فصل دوم: مبانی نظری:** در این فصل، مفاهیم و فناوری‌های بنیادین پروژه مانند بلاکچین، اتریوم، قراردادهای هوشمند، توکن‌های غیرقابل معاوضه (NFT)، استاندارد ERC-721، ذخیره‌سازی غیرمتمرکز با IPFS و عملکرد کیف پول‌های دیجیتال بررسی می‌شود.

¹ Sign-In with Ethereum

- **فصل سوم: نیازمندی‌ها:** این فصل به تعریف دقیق نیازمندی‌های پروژه اختصاص دارد که به دو دسته کاربردی¹ مانند ساخت و خرید توکن، و غیرکاربردی² مانند امنیت و پایداری سیستم تقسیم‌بندی می‌شوند.
- **فصل چهارم: معماری و طراحی سیستم:** در این فصل، معماری سطح بالای سیستم، پشته فناوری و طراحی دقیق اجزای مختلف سیستم شامل ساختار قرارداد هوشمند، شمای پایگاه داده، API‌های بک‌اند و جریان‌های کاربری در فرانت‌اند تشریح می‌گردد.
- **فصل پنجم: پیاده‌سازی:** در این فصل، جزئیات پیاده‌سازی و کدهای نوشته شده برای هر بخش ارائه می‌شود. قطعه کدهای مهم از قرارداد هوشمند، منطق API‌های بک‌اند، کامپوننت‌های فرانت‌اند و روند احراز هویت به شکل دقیق شرح داده می‌شوند.
- **فصل ششم: اجرا و تست:** این فصل به ارائه یک راهنمای گام‌به‌گام برای راه‌اندازی و اجرای کامل پروژه از ابتدا می‌پردازد. همچنین، سناریوهای تست طراحی شده برای اعتبارسنجی تمام قابلیت‌های کلیدی سیستم، از جمله ورود کاربر، ساخت توکن، خرید و فروش و تست‌های امنیتی را تشریح می‌کند.
- **فصل هفتم: نتیجه‌گیری و کارهای آینده:** در فصل پایانی، دستاوردهای پروژه در مقایسه با اهداف اولیه جمع‌بندی شده و در نهایت، پیشنهاداتی برای توسعه و افزودن قابلیت‌های جدید به پروژه در آینده ارائه می‌گردد.

¹ functional

² Non-functional

فصل دوم

مبانی نظری

این فصل به بررسی مفاهیم و فناوری‌های پایه‌ای می‌پردازد که اساس پروژه را تشکیل می‌دهند. درک این مبانی برای فهم معماری و فرآیندهای پیاده‌سازی شده در فصل‌های بعدی ضروری است.

۱-۲ بلاکچین

فناوری بلاکچین یک دفتر کل توزیع شده¹ و تغییرناپذیر است. در این فناوری، داده‌ها در قالب بلوک‌هایی ذخیره می‌شوند که از طریق توابع رمزنگاری به یکدیگر متصل شده و یک زنجیره را تشکیل می‌دهند. هر بلوک جدید، هش رمزنگاری شده‌ی بلوک قبلی را در خود جای می‌دهد که این امر، دستکاری یا حذف اطلاعات در بلوک‌های قدیمی را تقریباً غیرممکن می‌سازد.

¹ Distributed Ledger

مهم‌ترین ویژگی‌های بلاکچین عبارت‌اند از:

- **غیرمتمرکز:** برخلاف سیستم‌های سنتی که داده‌ها روی یک سرور مرکزی نگهداری می‌شوند، در بلاکچین، نسخه‌های یکسانی از دفتر روی کامپیوترهای سراسر شبکه توزیع شده است. این ویژگی، به امنیت سیستم در برابر مهاجمان کمک می‌کند.
- **شفافیت:** در بلاکچین، تمام تراکنش‌ها برای تمامی کاربران در شبکه قابل مشاهده است، که این مسئله به افزایش اعتماد در سیستم کمک می‌کند.
- **تغییرناپذیری:** پس از ثبت یک تراکنش در بلاکچین، تغییر یا حذف آن به دلیل ساختار زنجیره‌ای و مکانیزم‌های اجماع، بسیار دشوار و پرهزینه است.

۲-۲ قرارداد هوشمند

قرارداد هوشمند یک برنامه کامپیوتری است که بر روی بلاکچین ذخیره و اجرا می‌شود. این برنامه‌ها به صورت خودکار و بر اساس شرایط از پیش تعیین شده عمل می‌کنند و توابعی را بدون نیاز به واسطه اجرا می‌نمایند. در پروژه حاضر، قرارداد هوشمند ستون اصلی منطق روی زنجیره^۱ را تشکیل می‌دهد و مسئولیت مدیریت مالکیت توکن‌ها، فرآیندهای خرید و فروش و تضمین امنیت دارایی‌ها را بر عهده دارد.

۲-۳ اتریوم

اتریوم یک پلتفرم بلاکچین عمومی و متن‌باز است که مفهوم قراردادهای هوشمند را معرفی کرد و قابلیت‌های بلاکچین را از یک سیستم صرفاً برای تراکنش‌های مالی به یک کامپیوتر جهانی قابل برنامه‌ریزی ارتقا داد. در حالی که بیت‌کوین به عنوان نسل اول بلاکچین شناخته می‌شود، اتریوم نسل دوم آن محسوب می‌شود که به توسعه‌دهندگان اجازه می‌دهد برنامه‌های غیرمتمرکز^۲ را در این پلتفرم ایجاد و اجرا کنند.

نقطه قوت اصلی شبکه اتریوم، ماشین مجازی اتریوم^۳ است. این ماشین یک محیط اجرایی کاملاً ایزوله است که بر روی شبکه اتریوم قرار دارد و وظیفه اجرای کدهای قراردادهای هوشمند را بر عهده دارد. این ماشین مجازی تضمین می‌کند که یک قرارداد هوشمند، در همه سیستم‌های درون شبکه، نتیجه یکسانی خواهد داشت.

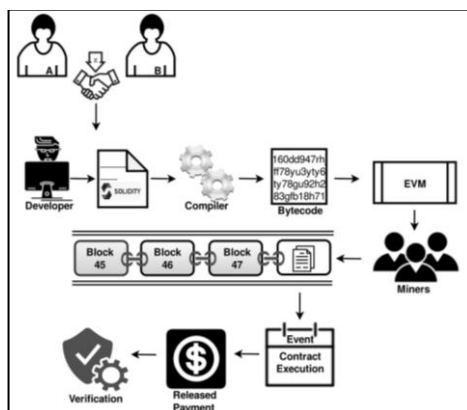
¹ On-chain

² Decentralized applications(dApps)

³ Ethereum virtual machine(EVM)

۲-۳-۱ Hardhat

یک محیط توسعه قدرتمند برای کامپایل، تست و استقرار قراردادهای هوشمند است. از Hardhat برای اجرای یک شبکه بلاکچین محلی^۱ جهت تست عملکرد قراردادها پیش از استقرار نهایی استفاده شده است. این شبکه به توسعه‌دهنده امکان تست قراردادها بدون هیچ هزینه‌ای را می‌دهد.



شکل ۲-۱: روند کلی پیاده‌سازی و اجرای قراردادهای هوشمند در شبکه اتریوم

۲-۴-۲ توکن‌های غیر قابل معاوضه (NFT)

NFT نوعی دارایی دیجیتال بر بستر بلاکچین است که نشان‌دهنده مالکیت یک آیتم یکتا (مانند عکس، یک موسیقی، یا آیتم در بازی) است. برخلاف ارزهای دیجیتال که قابل معاوضه هستند (هر واحد با واحد دیگر ارزش یکسان دارد)، هر توکن غیر قابل معاوضه دارای شناسه‌ای یکتا است که آن را از دیگری متمایز می‌کند. این ویژگی، امکان اثبات مالکیت دارایی‌های دیجیتال را فراهم کرده است. رشد بازار این توکن‌ها در سال‌های اخیر بسیار چشمگیر بوده و فرصت‌های جدیدی برای هنرمندان، تولیدکنندگان محتوا و مجموعه‌داران ایجاد کرده است.

۲-۴-۱ استاندارد ERC-721

برای اطمینان از سازگاری NFTها در اکوسیستم اتریوم، استانداردهایی تعریف شده‌اند. ERC-721 اولین و رایج‌ترین استاندارد برای ساخت NFT است. این استاندارد مجموعه‌ای از توابع را تعریف می‌کند که یک قرارداد هوشمند باید آن‌ها را پیاده‌سازی کند. توابع کلیدی این استاندارد شامل موارد زیر است:

- `ownerOf(tokenId)`: مالک یک توکن مشخص را برمی‌گرداند.
- `transferFrom(from, to, tokenId)`: مالکیت یک توکن را از یک آدرس به آدرس دیگر منتقل می‌کند.

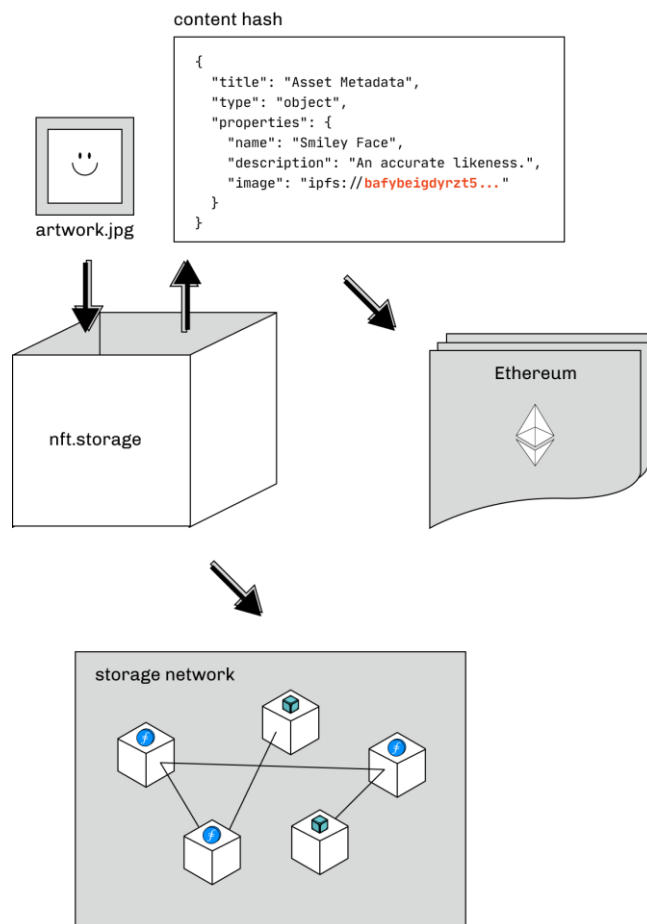
¹ Local

- `tokenURI(tokenId)`: یک آدرس اینترنتی^۱ را برمی گرداند که به یک فایل JSON حاوی متادیتا آن توکن اشاره می کند.

۵-۲ ذخیره سازی غیرمتمرکز NFT

یکی از چالش های اصلی در ساخت NFT، نحوه ذخیره سازی فایل های مرتبط با آن (مانند تصویر، توضیحات و ...) است. ذخیره سازی مستقیم این فایل ها بر روی بلاکچین بسیار پرهزینه و ناکارآمد است. هر کیلوبایت داده ای که روی بلاکچین اتریوم ذخیره می شود، هزینه قابل توجهی به شکل کارمزد تراکنش^۲ به دنبال دارد.

برای حل این مشکل، از راه حل های ذخیره سازی غیرمتمرکز استفاده می شود. در این روش، خود فایل دیجیتال در یک سیستم ذخیره سازی خارج از بلاکچین نگهداری شده و تنها یک لینک به آن در قرارداد هوشمند و NFT روی بلاکچین ثبت می شود.



شکل ۲-۲: نمودار ارتباطات بین اجزای مختلف در متد IPFS

^۱ Uniform Resource Identifier (URI)

^۲ Gas Fee

۲-۵-۱ پروتکل IPFS

IPFS یک پروتکل و شبکه هم‌تا به هم‌تا برای ذخیره و اشتراک گذاری فایل‌ها در یک سیستم فایل توزیع شده است. برخلاف وب سنتی که از آدرس‌دهی مبتنی بر مکان استفاده می‌کند (URL^۱ به مکان سرور اشاره می‌کند)، IPFS از آدرس‌دهی مبتنی بر محتوا (CID^۲ به جای URL) استفاده می‌کند. در این روش، هر فایل بر اساس محتوای خود یک هش رمزنگاری منحصر به فرد دریافت می‌کند که به عنوان آدرس آن فایل عمل می‌کند. یعنی اگر حتی یک پیکسل از فایل تغییر کند، CID آن به شکل کامل عوض می‌شود.

این متد مزایای مهمی برای NFT ها دارد:

- ماندگاری: تا زمانی که حداقل یک گره در شبکه IPFS فایل را نگهداری کند، آن فایل در دسترس خواهد بود و مشکل عدم دسترسی در وب سنتی را ندارد.
- یکپارچگی داده: از آنجایی که آدرس فایل، هش محتوای آن است، هرگونه تغییر در فایل منجر به تغییر آدرس آن می‌شود. این امر تضمین می‌کند که محتوای NFT پس از ایجاد، قابل دستکاری نخواهد بود.

۲-۶ کیف پول‌های دیجیتال و مدیریت هویت در Web3

در اکوسیستم Web3، هویت کاربر دیگر به حساب‌های کاربری متمرکز (مانند حساب گوگل) وابسته نیست. در عوض، هویت هر کاربر توسط کیف پول دیجیتال او مدیریت می‌شود.

یک کیف پول (مانند MetaMask) ابزاری است که کلیدهای عمومی و خصوصی کاربر را مدیریت می‌کند.

- کلید عمومی: به عنوان آدرس یا شناسه عمومی کاربر عمل می‌کند و می‌توان آن را بدون نگرانی با دیگران به اشتراک گذاشت.
- کلید خصوصی: یک رمز کاملاً محرمانه است که به کاربر اجازه می‌دهد به دارایی‌های خود دسترسی داشته باشد و تراکنش‌ها را امضا کند. این کلید هرگز نباید فاش شود.

¹ Uniform Resource Locator

² Content-ID

فصل سوم

نیازمندی‌ها

در این فصل، نیازمندی‌های پروژه به دو دسته اصلی کاربردی^۱ و غیرکاربردی^۲ تقسیم می‌شوند. نیازمندی‌های کاربردی به توصیف عملکردها و قابلیت‌هایی می‌پردازند که سیستم باید برای کاربران فراهم کند. این نیازمندی‌ها مشخص می‌کنند که سیستم «چه کارهایی» باید انجام دهد. نیازمندی‌های غیرکاربردی کیفیت و ویژگی‌های عملیاتی سیستم را تعریف می‌کنند. این نیازمندی‌ها مشخص می‌کنند که سیستم «چگونه» باید عمل کند.

۳-۱ نیازمندی‌های کاربردی

نیازمندی‌های کاربردی این پلتفرم بر اساس تعاملات کاربران مختلف با سیستم، از جمله خریداران، فروشندگان و سازندگان توکن، تعریف شده‌اند. این نیازمندی‌ها به شرح زیر دسته‌بندی می‌شوند:

^۱ Functional

^۲ Non-functional

۳-۱-۱ مدیریت کاربر و احراز هویت

- کاربر باید بتواند با استفاده از کیف پول MetaMask به سیستم وارد شود و با امضای دیجیتال خود، خود را احراز هویت کند.
- پس از اولین ورود موفق کاربر، سیستم باید به صورت خودکار یک پروفایل برای او در پایگاه داده ایجاد کرده و یک جلسه^۱ امن برای عملیات‌های بعدی برقرار کند.
- کاربر باید بتواند به صفحه پروفایل شخصی خود دسترسی داشته باشد و لیست توکن‌هایی مالک آنهاست یا برای فروش قرار داده و تاریخچه تراکنش‌های خود را مشاهده کند.

۳-۱-۲ چرخه حیات توکن

- کاربر احراز هویت شده باید بتواند یک توکن جدید ایجاد کند. این فرآیند شامل بارگذاری عکس، وارد کردن عنوان، توضیحات، انتخاب دسته‌بندی و تعیین قیمت فروش است. پس از پرداخت کارمزد به مقدار ۱ درصد قیمت فروش، توکن در بلاکچین ساخته شده و مالکیت آن به قرارداد هوشمند منتقل می‌گردد تا برای فروش در دسترس قرار گیرد.
- کاربر باید بتواند با مراجعه به صفحه جزئیات یک توکن، مشخصات آن را مشاهده کرده و با پرداخت مبلغ مشخص شده، مالکیت آن را از قرارداد هوشمند به کیف پول خود منتقل کند.
- کاربری که یک توکن را برای فروش قرار داده است، باید بتواند از طریق پروفایل خود فروش آن را لغو کند. پس از پرداخت کارمزد مربوطه، مالکیت توکن از قرارداد هوشمند به کیف پول کاربر بازگردانده می‌شود.
- کاربری که مالک یک توکن است، باید بتواند آن را برای فروش لیست کند. این فرآیند شامل تعیین قیمت جدید، تعیین دسته‌بندی و انتقال مالکیت توکن به قرارداد هوشمند است.

۳-۱-۳ عملیات مالی

- پس از فروش موفق یک توکن، مبلغ فروش پس از کسر کارمزد، به موجودی فروشنده در سیستم اضافه می‌شود.
- کاربر باید بتواند در هر زمان از طریق پروفایل خود، موجودی خود را به آدرس کیف پول متصل شده‌اش منتقل کند.

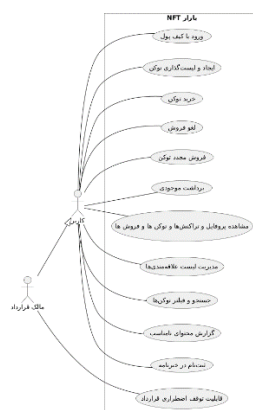
¹ Session

۳-۱-۴ تعاملات کاربری و بازار

- سیستم باید یک نوار جستجو فراهم کند که به کاربران اجازه دهد توکن‌ها را بر اساس کلمات کلیدی در عنوان و توضیحات آن‌ها جستجو کنند.
- کاربران باید بتوانند در صفحه بازار، لیست توکن‌ها را بر اساس دسته‌بندی و محدوده قیمت فیلتر کنند.
- کاربر وارد شده باید بتواند توکن‌های مورد علاقه خود را به یک لیست شخصی اضافه کرده و بعداً از طریق دکمه در سربرگ^۱ سایت خود به آن دسترسی داشته باشد.
- کاربران باید بتوانند یک توکن را به دلیل محتوای نامناسب یا مشکوک، به همراه ارائه توضیحات، گزارش دهند.
- در صفحه جزئیات هر توکن، باید گزینه‌هایی برای اشتراک‌گذاری لینک آن در شبکه‌های اجتماعی رایج وجود داشته باشد.
- کاربران باید بتوانند با وارد کردن آدرس ایمیل خود در بخش مربوطه، در خبرنامه پلتفرم عضو شوند.

۳-۱-۵ مدیریت قرارداد و امنیت

- مالک قرارداد هوشمند باید این قابلیت را داشته باشد که عملکردهای اصلی و حساس قرارداد را در مواقع ضروری به صورت موقت متوقف کند. این ویژگی به عنوان یک مکانیزم امنیتی برای محافظت از دارایی‌های کاربران در برابر تهدیدات احتمالی عمل می‌کند.



شکل ۳-۱: نمودار موارد استفاده^۲ پروژه

^۱ Header

^۲ Use case diagram

۲-۳ نیازمندی‌های غیر کاربردی

نیازمندی‌های غیر کاربردی این پلتفرم بر اساس امنیت، کارایی، قابلیت اطمینان و تجربه کاربری تعریف شده‌اند. این نیازمندی‌ها به شکل زیر دسته‌بندی می‌شوند:

۱-۲-۳ امنیت

- احراز هویت باید مبتنی بر اثبات مالکیت کلید باشد و بدون وابستگی به رمز عبور، یک نشست معتبر برای دسترسی به API ایجاد کند.
- سامانه باید در برابر حملات بازورود^۱ مقاوم باشد و هیچ عملیات مالی حساس قابل تکرار در یک تراکنش نباشد.
- دسترسی به عملیات حساس باید مبتنی بر نقش/مالکیت کنترل شود؛ فقط صاحبان مجاز قادر به اجرای این توابع باشند.
- همه ورودی‌ها در لایه‌ی قرارداد و سرور باید اعتبارسنجی شوند؛ درخواست‌های نامعتبر رد شوند.

۲-۲-۳ اطمینان و استحکام

- سامانه باید مکانیزم توقف اضطراری برای تعلیق عملیات حساس داشته باشد.
- عملیات داده‌ای چندجدولی باید اتمیک باشد و در صورت خطا، تغییرات به‌طور کامل بازگردند.
- خطاها باید با کدهای وضعیت استاندارد و پیام‌های معنادار برگشت داده شوند.

۳-۲-۳ تجربه کاربری و سازگاری

- کاربر باید بتواند به آسانی با استفاده از کیف پول خود، به سایت وارد و احراز هویت شود.
- کارمزدهای عملیات مختلف باید به طور شفاف به کاربر اطلاع‌رسانی بشوند.
- تمام توکن‌های ایجاد شده باید با استانداردهای رایج اکوسیستم اتریوم (ERC-721) سازگار باشند.
- سامانه باید روی شبکه‌های تست/اصلی قابل استقرار و استفاده باشد.

¹ Re-entrancy

فصل چهارم

معماری و طراحی سیستم

پس از تعریف نیازمندی‌ها در فصل گذشته، این فصل به تشریح ساختار فنی، معماری و جزئیات طراحی سیستم می‌پردازد. در این فصل، ابتدا معماری کلی سیستم از نمای کلان بررسی شده، سپس پشته فناوری^۱ و دلایل انتخاب هر ابزار تشریح می‌شود و در نهایت، طراحی دقیق هر یک از اجزای اصلی سیستم ارائه می‌گردد.

۴-۱ معماری سطح بالا

معماری این پلتفرم از یک رویکرد ترکیبی استفاده می‌کند که از نقاط قوت سیستم‌های متمرکز و غیرمتمرکز به طور همزمان بهره می‌برد. این سیستم به عنوان یک برنامه غیرمتمرکز با معماری چندلایه طراحی شده است که منطق آن بین اجزای روی زنجیره و خارج از زنجیره^۲ پخش شده است. سیستم از سه لایه اصلی تشکیل شده است که هر کدام وظایف مشخصی را بر عهده دارند:

^۱ Tech stack

^۲ Off-chain

۱. **لایه ارائه (فرانت‌اند):** این لایه مسئولیت نمایش رابط کاربری به کاربر نهایی و مدیریت تعاملات او با سیستم را بر عهده دارد. این لایه در مرورگر کاربر اجرا می‌شود.

مسئولیت‌های اصلی این لایه عبارتند از:

- **ارائه رابط کاربری:** نمایش صفحات مختلف سیستم مانند صفحه اصلی، جزئیات توکن و پروفایل کاربر.
- **مدیریت وضعیت سمت کلاینت:** مدیریت داده‌های موقتی مانند اطلاعات فرم‌ها و وضعیت بارگذاری داده‌ها. این لایه همچنین مسئولیت مدیریت بهینه داده‌های دریافتی از سرور و بلاکچین را برای بهبود عملکرد بر عهده دارد.
- **تعامل با کیف پول کاربر:** برقراری ارتباط با کیف پول‌های مبتنی بر مرورگر برای دریافت آدرس عمومی کاربر، درخواست امضای دیجیتال و ارسال تراکنش‌ها به شبکه بلاکچین.

۲. **لایه منطق:** این لایه شامل توابع و مسائل اصلی سیستم است و خود به دو بخش تقسیم می‌شود:

- **منطق خارج از زنجیره (بک‌اند):** این بخش مسئولیت پردازش‌هایی را بر عهده دارد که اجرای آن‌ها روی بلاکچین ضروری یا بهینه نیست.

وظایف این لایه عبارتند از:

- **مدیریت احراز هویت و جلسات:** اعتبارسنجی هویت کاربران از طریق امضای دیجیتال کیف پول آن‌ها و مدیریت جلسات ورود.
- **تعامل با پایگاه داده:** ذخیره و بازیابی اطلاعاتی که نیازی به ثبت روی بلاکچین ندارند (مانند پروفایل‌های کاربری، دسته‌بندی‌ها و گزارش‌ها).
- **ارتباط با سرویس‌های خارجی:** مدیریت آپلود و ذخیره‌سازی فایل‌ها بر روی یک سیستم ذخیره‌سازی غیرمتمرکز (IPFS).
- **ارائه داده طبق درخواست کاربر:** پیاده‌سازی قابلیت‌های پیچیده‌تر مانند جستجو، فیلتر و صفحه‌بندی داده‌ها.

- **منطق روی زنجیره (قرارداد هوشمند):** این بخش، بخش غیرمتمرکز و قابل اعتماد سیستم است که با زبان Solidity نوشته شده و بر روی بلاکچین مستقر می‌شود. مسئولیت‌های آن عبارتند از:

○ **مدیریت چرخه حیات توکن:** اجرای توابع ساخت، انتقال مالکیت و ثبت متادیتا (TokenURI).

○ **اجرای منطق بازار:** مدیریت فرآیندهای لیست کردن برای فروش، خرید و لغو فروش.

○ **مدیریت موجودی و کارمزدها:** نگهداری امن مبالغ فروش تا زمان برداشت توسط فروشنده و محاسبه و تعیین کارمزد هر توکن.

۳. **لایه داده:** این لایه نیز به دو بخش تقسیم می‌شود:

• **ذخیره‌سازی خارج از زنجیره:** یک پایگاه داده بر پایه SQLite که از طریق Prisma ORM مدیریت می‌شود.

• **ذخیره‌سازی غیرمتمرکز:** فایل‌های اصلی توکن‌ها و متادیتا آن‌ها بر روی IPFS ذخیره می‌شوند.

۲-۴ فناوری‌های استفاده شده

در این بخش، ابزارهای اصلی استفاده شده در سه لایه اصلی پروژه (فرانت‌اند، بک‌اند و قرارداد هوشمند) معرفی شده و دلیل انتخاب هر یک تشریح می‌شود:

• قرارداد هوشمند:

○ **Solidity:** زبان برنامه‌نویسی متداول برای توسعه قراردادهای هوشمند بر روی بلاکچین اتریوم و ماشین مجازی اتریوم است. با توجه به قدرت و جامعیت این زبان، به عنوان انتخاب اصلی برای پیاده‌سازی منطق اصلی پروژه انتخاب شد.

○ **OpenZeppelin:** مجموعه‌ای از قراردادهای هوشمند استاندارد و امن است که به صورت گسترده توسط توسعه‌دهندگان بلاکچین استفاده می‌شود. این کتابخانه به دلیل ارائه پیاده‌سازی‌های امن و بهینه از استانداردهای رایج توکن (مانند ERC721) و الگوهای امنیتی (مانند کنترل دسترسی و جلوگیری از حملات بازورودی)، ریسک‌های امنیتی را کاهش داده و فرآیند توسعه را سریع می‌کنند.

• فرانت‌اند و بک‌اند:

○ **React:** به دلیل اکوسیستم گسترده و مدل برنامه‌نویسی مبتنی بر مولفه^۱، ابزاری قدرتمند برای ساخت رابط‌های کاربری پویا می‌باشد.

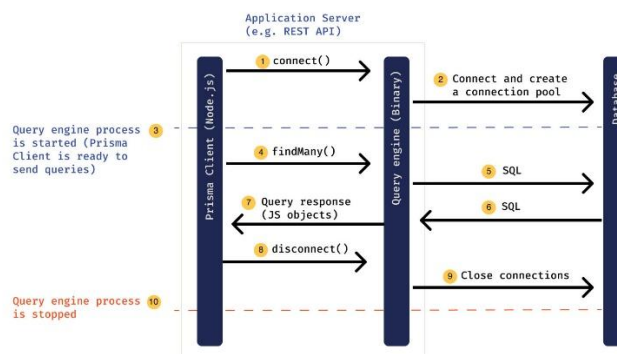
¹ Component

○ **Next.js:** یک چارچوب نرم‌افزاری مبتنی بر کتابخانه React است که برای ساخت برنامه‌های وب سریع، بهینه و مقیاس‌پذیر طراحی شده است. دلیل اصلی انتخاب این چارچوب برای پروژه حاضر، ساختار یکپارچه فرانت‌اند و بک‌اند آن است. Next.js به توسعه‌دهندگان اجازه می‌دهد تا در یک پروژه واحد، هم رابط کاربری سمت کاربر را توسعه دهند و هم منطق سمت سرور را از طریق مسیرهای API پیاده‌سازی کنند. این ویژگی، پیچیدگی مدیریت دو پروژه مجزا برای فرانت‌اند و بک‌اند را از بین می‌برد. علاوه بر این، قابلیت‌های پیشرفته‌ای مانند رندر سمت سرور^۱، به افزایش سرعت بارگذاری صفحات کمک می‌کند.

○ **Node.js و Next.js API Routes:** منطق بک‌اند در محیط Node.js و با استفاده از مسیرهای API در Next.js توسعه داده شد. این رویکرد یکپارچه، نیاز به راه‌اندازی و مدیریت یک سرور را از بین برده و فرآیند توسعه را ساده‌تر و سریع‌تر می‌کند.

• پایگاه داده:

○ **Prisma:** یک ORM^۲ برای محیط Node.js است. نقش اصلی آن، ساده‌سازی و ایمن‌سازی فرآیند کار با پایگاه داده است. توسعه‌دهنده به جای نوشتن دستورات SQL خام که در آن‌ها امکان خطا و آسیب‌پذیری وجود دارد، مدل پایگاه داده^۳ را در یک فایل شمای Prisma تعریف می‌کند. سپس Prisma به برنامه اجازه می‌دهد با استفاده از توابع ساده با پایگاه داده کار کند. این رویکرد نه تنها سرعت توسعه را افزایش می‌دهد، بلکه با اعتبارسنجی خودکار کوئری‌ها، از بروز بسیاری از خطاهای رایج در کار با پایگاه داده جلوگیری می‌کند.



شکل ۴-۱: نمودار ارتباطات و پیام‌ها در Prisma

^۱ Server-Side Rendering(SSR)

^۲ Object-relational mapping

^۳ Database

- **SQLite:** به دلیل سادگی، عدم نیاز به سرور مجزا و ذخیره سازی پایگاه داده در یک فایل واحد، برای فاز توسعه و پروژه اولیه مناسب است.

• تعامل با بلاکچین:

- **Viem:** یک رابط سبک، سریع و امن برای تعامل با بلاکچین اتریوم است. این کتابخانه وظیفه اصلی ارسال درخواست ها به بلاکچین و قالب بندی داده های دریافتی را بر عهده دارد.
- **Wagmi:** مجموعه ای از هوک های React است که بر پایه viem ساخته شده است. با استفاده از این کتابخانه، کارهایی مانند اتصال به کیف پول و دریافت آدرس و موجودی کاربر به راحتی قابل انجام هستند.
- **RainbowKit:** این ابزار که بر روی wagmi ساخته شده، کتابخانه خوبی برای ساخت رابط کاربری اتصال کیف پول هست که تمام منطق مورد نیاز برای اتصال به کیف پول های مختلف، نمایش اطلاعات کاربر و مدیریت شبکه های بلاکچین را در خود جای داده است.

• احراز هویت:

- **NextAuth و SIWE:** برای ورود امن و مبتنی بر Web3، از ترکیب NextAuth و پروتکل SIWE استفاده شده است. این روش به کاربران اجازه می دهد تا تنها با امضای یک پیام از طریق کیف پول خود، هویتشان را اثبات کنند و نیازی به رمز عبور سنتی ندارند.

۳-۴ طراحی اجزای سیستم

در این بخش، طراحی داخلی هر یک از اجزای اصلی سیستم با جزئیات بیشتری تشریح می شود.

۱-۳-۴ طراحی قرارداد هوشمند

قرارداد هوشمند NFTMarketplace پایه و اساس منطق روی زنجیره پروژه است و وظیفه مدیریت دارایی ها و تعاملات مالی را بر عهده دارد. این قرارداد با ارث بری از استانداردهای OpenZeppelin طراحی شده است تا از امنیت و کارایی بالایی برخوردار باشد.

• ساختار اصلی:

- قرارداد بر پایه استاندارد توکن غیر قابل معاوضه ERC721 طراحی شده است تا با اکوسیستم اتریوم سازگار باشد.

¹ Hook

- برای هر آیتم قابل فروش در بازار، یک ساختار داده اختصاصی طراحی شده است که اطلاعات کلیدی مانند شناسه توکن، آدرس فروشنده، قیمت و وضعیت فروش را نگهداری می کند.

رویدادها

برای امکان ردیابی فعالیت ها توسط برنامه های خارج از زنجیره، رویدادهایی تعریف می شوند تا فعالیت هایی مانند لیست توکن جدید، فروش موفق و برداشت وجه توسط کاربر را به خارج از زنجیره اطلاع دهند.

• سیاست های امنیتی

امنیت در قراردادهای هوشمند از اهمیت بالایی برخوردار است. در این پروژه از مکانیزم های زیر استفاده شده است:

- **Ownable2Step**: این قرارداد تضمین می کند که توابع حساس مدیریتی مانند pause/unpause فقط توسط مالک قرارداد قابل فراخوانی هستند. فرآیند انتقال مالکیت دو مرحله ای نیز از انتقال اشتباهی مالکیت جلوگیری می کند.
- **ReentrancyGuard**: توابع حساس که با انتقال اتر^۱ همراه هستند، مانند createMarketSale, cancelListing و withdraw با این کتابخانه محافظت می شوند.
- **Pausable**: این قابلیت به مالک قرارداد اجازه می دهد تا در شرایط اضطراری، عملکردهای اصلی مانند ساخت، خرید و فروش مجدد توکن را متوقف کند تا از دارایی کاربران محافظت شود.

• توابع

قرارداد هوشمند شامل مجموعه ای از توابع است که عملکردهای اصلی بازار را مدیریت می کنند. این توابع به سه دسته اصلی تقسیم می شوند:

۱. توابع اصلی بازار

این دسته از توابع، مسئولیت مدیریت چرخه کامل حیات یک توکن غیرقابل معاوضه در بازار را بر عهده دارند. طراحی این بخش شامل قابلیت های زیر است:

- **ایجاد و عرضه اولیه توکن**: سیستمی طراحی شده است که به کاربر اجازه می دهد یک توکن جدید بسازد و آن را برای فروش لیست کند. در این فرآیند، اطلاعات توکن (متادیتا) ثبت شده و مالکیت آن به صورت موقت به قرارداد هوشمند منتقل می شود تا برای فروش در دسترس قرار گیرد.

¹ Ether

- **خرید توکن:** تابعی برای مدیریت فرآیند خرید طراحی شده است. این تابع، مبلغ پرداختی خریدار را با قیمت تعیین شده مطابقت می دهد و در صورت تایید، مالکیت توکن را به کیف پول خریدار منتقل می کند.
- **فروش مجدد توکن:** برای مالکان توکن ها این امکان طراحی شده است که بتوانند دارایی خود را مجدداً با قیمت جدید برای فروش لیست کنند. این فرآیند نیز شامل انتقال موقت مالکیت به قرارداد هوشمند برای تضمین امنیت معامله است.
- **لغو فروش:** فروشنده این قابلیت را خواهد داشت که فروش یک توکن را لغو کند. در این صورت، مالکیت توکن از قرارداد هوشمند به کیف پول فروشنده بازگردانده می شود.

۲. توابع مالی و مشاهده اطلاعات

این بخش از طراحی بر مدیریت دارایی ها و شفافیت اطلاعات متمرکز است:

- **مدیریت و برداشت درآمد:** مکانیزمی طراحی شده است که درآمد حاصل از فروش توکن ها پس از کسر کارمزد، در یک موجودی داخلی برای فروشنده نگهداری شود. فروشندگان می توانند در هر زمان، موجودی قابل برداشت خود را به کیف پول شخصی شان منتقل کنند.
- **ارائه اطلاعات به کاربر:** توابعی برای ارائه اطلاعات شفاف به کاربران طراحی شده است. کاربران می توانند موجودی قابل برداشت خود و همچنین لیستی از توکن هایی که در مالکیت دارند را از قرارداد هوشمند استعلام کنند.

۳. توابع مدیریتی

برای اطمینان از امنیت و کنترل پلتفرم، توابع مدیریتی خاصی طراحی شده اند که تنها توسط مالک قرارداد قابل فراخوانی هستند:

- **قابلیت توقف اضطراری:** یک مکانیزم امنیتی برای قرارداد طراحی شده است که به مالک اجازه می دهد در شرایط بحرانی یا برای بروزرسانی، عملکردهای اصلی بازار مانند خرید و فروش را به طور موقت متوقف یا مجدداً فعال کند. این ویژگی از دارایی کاربران در برابر تهدیدات احتمالی محافظت می کند.

۴-۳-۲ طراحی بک اند

بک اند این پروژه به عنوان یک لایه خدماتی طراحی شده است که وظیفه مدیریت داده های خارج از زنجیره و ارتباط با سرویس های دیگر را بر عهده دارد.

• **مدل‌های داده^۱:** برای مدیریت داده‌های خارج از زنجیره، موجودیت‌های اصلی زیر طراحی شده‌اند:

- **کاربر:** برای ذخیره اطلاعات پروفایل کاربران.
- **توکن:** برای نگهداری اطلاعات تکمیلی توکن‌ها مانند نام، توضیحات و ارجاع به فایل‌های ذخیره شده.
- **لیست فروش:** برای ثبت اطلاعات مربوط به هر آیتمی که برای فروش گذاشته می‌شود.
- **تراکنش:** برای ثبت تاریخچه خرید و فروش‌ها.
- سایر مدل‌ها برای پشتیبانی از قابلیت‌هایی مانند لیست علاقه‌مندی‌ها، گزارش تخلف و خبرنامه.

• **API:**

بک‌اند برای پشتیبانی از عملیات‌های سمت کاربر طراحی شده است. مسئولیت‌های اصلی این API عبارتند از:

- **مسیرهای احراز هویت:** مدیریت فرآیند ورود و خروج کاربران.
- **مسیرهای مدیریت توکن:** پشتیبانی از فرآیندهای ساخت توکن، آپلود فایل‌ها و ذخیره اطلاعات در پایگاه داده.
- **مسیرهای بازار:** ارائه داده‌های مربوط به توکن‌های موجود در بازار با قابلیت فیلتر و جستجو.
- **مسیرهای پروفایل کاربر:** ارائه اطلاعات شخصی‌سازی شده برای هر کاربر، مانند توکن‌های تحت مالکیت و تاریخچه فعالیت‌ها.
- **مسیرهای پشتیبانی:** مدیریت قابلیت‌هایی مانند لیست علاقه‌مندی‌ها، گزارش تخلف و ثبت نام در خبرنامه.

۳-۳-۴ طراحی فرانت‌اند

رابط کاربری با هدف ارائه یک تجربه ساده و روان طراحی شده است.

• **نماهای اصلی:** رابط کاربری شامل چندین نمای اصلی برای پوشش نیازهای کاربر است:

- **نمای خانه:** نقطه ورود کاربر به پلتفرم و نمایش کلی محتوا.

¹ Data models

- **نمای بازار:** فضایی برای نمایش تمام توکن‌های قابل فروش همراه با ابزارهای جستجو و فیلتر.
- **نمای ساخت توکن:** یک فرم تعاملی برای راهنمایی کاربر در فرآیند ایجاد یک توکن جدید.
- **نمای جزئیات توکن:** صفحه‌ای اختصاصی برای نمایش اطلاعات کامل هر توکن.
- **نمای پروفایل کاربری:** یک داشبورد شخصی برای مدیریت دارایی‌ها و مشاهده تاریخچه فعالیت‌ها.

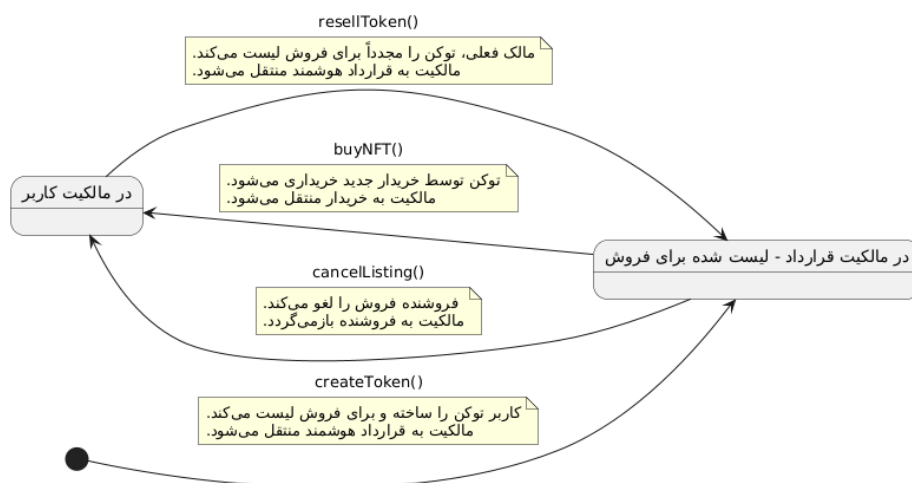
• مدیریت وضعیت

برای مدیریت داده‌ها و وضعیت کلی برنامه در سمت کلاینت، از یک رویکرد متمرکز استفاده می‌شود. یک لایه مدیریت وضعیت سراسری طراحی شده است که وظیفه پیکربندی ابزارهای ارتباطات با بلاکچین و مدیریت جلسات کاربر را بر عهده دارد. علاوه بر این، یک بستر اشتراک داده اختصاصی برای پروژه طراحی شده تا توابع مربوط به تعامل با قرارداد هوشمند را به صورت بهینه در اختیار تمام اجزای رابط کاربری قرار دهد.

۴-۴ چرخه حیات توکن

هر توکن غیرقابل معاوضه در این پلتفرم، یک چرخه حیات مشخص دارد که وضعیت آن را از لحظه ایجاد تا فروش نهایی یا لغو فروش تعریف می‌کند. این چرخه حیات مستقیماً توسط منطق پیاده‌سازی شده در قرارداد هوشمند مدیریت می‌شود. یک توکن می‌تواند در یکی از دو وضعیت اصلی زیر قرار داشته باشد:

1. **در مالکیت کاربر:** در این وضعیت، توکن در کیف پول یک کاربر قرار دارد و او مالک قطعی آن است.
2. **لیست شده برای فروش:** در این وضعیت، توکن برای فروش در بازار قرار گرفته است. برای تضمین امنیت و اجرای خودکار فرآیند فروش، مالکیت توکن به صورت موقت به آدرس خود قرارداد هوشمند منتقل می‌شود.



شکل ۴-۲: نمودار ماشین حالت چرخه حیات توکن

فصل پنجم

پیاده‌سازی

این فصل به تشریح جزئیات پیاده‌سازی و کدهای نوشته‌شده برای هر یک از اجزای اصلی پروژه، یعنی قرارداد هوشمند، بک‌اند و فرانت‌اند، می‌پردازد. هدف این بخش، نمایش چگونگی تبدیل طراحی‌های تئوری به یک محصول نرم‌افزاری کاربردی از طریق کدنویسی است. در هر بخش، قطعه کدهای کلیدی برای درک بهتر منطق پیاده‌سازی شده، ارائه می‌گردد.

۵-۱ پیاده‌سازی قرارداد هوشمند

قرارداد هوشمند با استفاده از زبان Solidity نسخه 0.8.28 و چارچوب توسعه Hardhat پیاده‌سازی شده است. برای تضمین امنیت و استانداردسازی، این قرارداد از مازول‌های کتابخانه OpenZeppelin ارث‌بری می‌کند.

۵-۱-۱ ساختار اصلی و متغیرهای قرارداد

ساختار اصلی قرارداد شامل متغیرهای حالت برای مدیریت توکن‌ها و وجوه است:

- **MarketItem**: یک ساختار که اطلاعات کلیدی هر NFT را نگهداری می‌کند. این اطلاعات شامل شناسه توکن (tokenId)، آدرس فروشنده، آدرس مالک فعلی، قیمت و وضعیت فروش است.
- **_tokenIds**: یک شمارنده خصوصی برای ایجاد شناسه‌های یکتای جدید برای هر توکن.
- **FEE_BPS**: یک ثابت عمومی که کارمزد پلتفرم را بر اساس واحد پایه تعریف می‌کند (۱۰۰ واحد معادل ۱٪).
- **Balances**: یک mapping که موجودی اتر قابل برداشت هر فروشنده را ذخیره می‌کند.
- **idMarketItem**: یک mapping که هر tokenId را به یک MarketItem متناظر آن مرتبط می‌سازد و دسترسی سریع به اطلاعات فروش را فراهم می‌کند.

۵-۲ توابع اصلی قرارداد

توابع قرارداد هوشمند به سه دسته اصلی تقسیم می‌شوند:

۱. توابع اصلی بازار: این توابع چرخه حیات یک توکن غیرقابل معاوضه را در بازار مدیریت می‌کنند:

- **createMarketItem+createToken**: تابع createToken به کاربران اجازه می‌دهد تا یک توکن جدید ایجاد کرده و آن را مستقیماً برای فروش لیست کنند. در این فرآیند، یک tokenId جدید تولید شده، توکن برای کاربر mint می‌شود و tokenId آن ثبت می‌گردد. سپس با فراخوانی تابع createMarketItem، مالکیت توکن به قرارداد هوشمند منتقل شده و برای فروش لیست می‌شود. برای انجام این عملیات، کاربر باید کارمزد لیست کردن را پرداخت کند.
- **createMarketSale**: این تابع فرآیند خرید یک توکن را مدیریت می‌کند. تابع ابتدا بررسی می‌کند که خریدار، خود فروشنده نباشد و قیمت ارسالی با قیمت ثبت شده برای توکن برابر باشد. در صورت موفقیت، مبلغ معامله پس از کسر کارمزد به موجودی فروشنده اضافه شده و مالکیت توکن به خریدار منتقل می‌شود.
- **resellToken**: کاربری که مالک یک توکن است، می‌تواند با استفاده از این تابع، آن را مجدداً برای فروش با قیمت جدیدی لیست کند. پیش از فراخوانی این تابع، کاربر باید به قرارداد هوشمند اجازه انتقال توکن را از طرف خود داده باشد.
- **cancelListing**: فروشنده می‌تواند با فراخوانی این تابع، فروش یک توکن را لغو کند. در این صورت، توکن از حالت فروش خارج شده و مالکیت آن از قرارداد هوشمند به کیف پول فروشنده بازگردانده می‌شود.

۲. توابع مالی و مشاهده اطلاعات:

- **withdraw**: این تابع به فروشندگان اجازه می‌دهد تا درآمد حاصل از فروش توکن‌های خود را که در متغیر `balances` ذخیره شده است، به کیف پول شخصی خود منتقل کنند.
- **fetchMyNFTs**: این تابع تمام توکن‌هایی را که در مالکیت کیف پول متصل شده به قرارداد قرار دارند، باز می‌گرداند و در صفحه پروفایل کاربر استفاده می‌شود.

۳. توابع مدیریتی و امنیتی:

- **Pausable**: با ارث‌بری از این کتابخانه `OpenZeppelin`، مالک قرارداد می‌تواند در شرایط اضطراری، عملکردهای حساس مانند ساخت، خرید و فروش مجدد توکن را متوقف کند.
- **ReentrancyGuard**: توابع حساس که با انتقال اتر^۱ مانند `cancelListing`، `createMarketSale` و `withdraw` با `modifier` به نام `nonReentrant` محافظت شده‌اند تا از حملات بازورود جلوگیری شود.

۲-۵ پیاده‌سازی بک‌اند

بک‌اند پروژه با استفاده از مسیرهای `API` در `Next.js` پیاده‌سازی شده است. این رویکرد یکپارچه، نیاز به راه‌اندازی یک سرور مجزا را از بین برده و فرآیند توسعه را ساده‌تر می‌کند. بک‌اند مسئولیت مدیریت داده‌های خارج از زنجیره، احراز هویت کاربران و ارتباط با سرویس‌های خارجی مانند `IPFS` را بر عهده دارد.

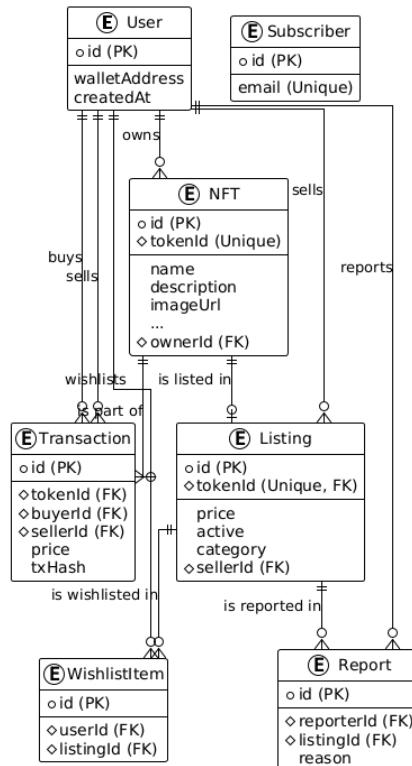
۱-۲-۵ شمای پایگاه داده:

ساختار پایگاه داده با استفاده از `Prisma` تعریف شده است. مدل‌های اصلی داده به شرح زیر هستند:

- **User**: اطلاعات کاربران را بر اساس آدرس کیف پول آن‌ها ذخیره می‌کند.
- **NFT**: اطلاعات اصلی هر توکن غیرقابل معاوضه، شامل `tokenId`، نام، توضیحات، آدرس تصویر و متادیتا را نگهداری می‌کند.
- **Listing**: اطلاعات مربوط به لیست‌های فروش فعال، شامل قیمت، دسته‌بندی و شناسه فروشنده را ذخیره می‌کند.
- **Transaction**: تاریخچه‌ای از تمام تراکنش‌های خرید و فروش را ثبت می‌کند.

¹ Ether

- **WishlistItem**: ارتباط بین کاربران و لیست‌های مورد علاقه آن‌ها را مدیریت می‌کند.
- **Report**: گزارش‌های ثبت‌شده توسط کاربران را ذخیره می‌کند.
- **Subscriber**: ایمیل‌های ثبت‌شده برای خبرنامه را نگهداری می‌کند.



شکل ۵-۱: نمودار ارتباط بین موجودیت‌های پایگاه داده

۵-۲-۲ پیاده‌سازی API

در پیاده‌سازی API‌های مرتبط با پایگاه‌داده، استفاده از `prisma.$transaction` حیاتی است، زیرا تضمین می‌کند که تمام عملیات (به‌روزرسانی مالکیت، غیرفعال کردن لیست فروش و ثبت تراکنش) به صورت اتمیک انجام شوند. اگر هر یک از این مراحل با خطا مواجه شود، تمام تغییرات لغو (Rollback) می‌گردد تا از عدم سازگاری داده‌ها جلوگیری شود.

مسیرهای API این پروژه عبارتند از:

- `/api/auth/[...nextauth]`: مدیریت فرآیندهای احراز هویت مبتنی بر SIWE.
- `/api/create-nft/upload-image`: آپلود فایل تصویر بر روی IPFS از طریق سرویس Pinata.
- `/api/create-nft/create-metadata`: ایجاد و آپلود فایل متادیتای JSON بر روی IPFS.

فرمت متادیتا به این شکل است:

```
{"name":"nft1","description":"desc1","image":"ImageURI"}
```

- **/api/create-nft/save-nft**: ذخیره اطلاعات توکن جدید در پایگاه داده.
- **/api/buy-nft**: به روزرسانی وضعیت توکن در پایگاه داده پس از خرید.
- **/api/resell**: به روزرسانی یا ایجاد لیست فروش جدید برای یک توکن در پایگاه داده.
- **/api/cancel-sell**: غیرفعال کردن لیست فروش و بازگرداندن مالکیت توکن به کاربر در پایگاه داده.
- **/api/profile**: دریافت اطلاعات پروفایل کاربر، شامل لیست توکن‌های فروشی و تاریخچه تراکنش‌ها.
- **/api/fetch-market-items**: دریافت لیست توکن‌های موجود در بازار با قابلیت جستجو و فیلتر.
- **/api/wishlist**: افزودن، حذف و بررسی وضعیت توکن‌ها در لیست علاقه‌مندی‌های کاربر.
- **/api/report**: ثبت گزارش تخلف برای یک توکن.
- **/api/subscribe**: مدیریت ثبت‌نام کاربران در خبرنامه.

۳-۵ پیاده‌سازی فرانت‌اند

فرانت‌اند پروژه با استفاده از فریم‌ورک Next.js و کتابخانه React توسعه داده شده است.

۱-۳-۵ ساختار و صفحات اصلی

- **صفحه اصلی (/):** شامل معرفی کلی، نمایش توکن‌های تصادفی و دسته‌بندی‌ها.
- **بازار (/marketplace):** در این صفحه، لیست توکن‌ها با قابلیت فیلتر و صفحه‌بندی نمایش داده می‌شود. وضعیت فیلترها از طریق پارامترهای URL مدیریت شده و با استفاده از `useSearchParams` خوانده می‌شود.
- **ساخت توکن (/create-nft):** این صفحه شامل فرم ورود اطلاعات، قیمت و کامپوننت `DropZone` برای آپلود تصویر است. منطق این صفحه، فرآیند چند مرحله‌ای ساخت توکن را مدیریت می‌کند.
- **صفحه پروفایل (/profile/page.jsx):** این صفحه به کاربران احراز هویت شده اجازه می‌دهد تا توکن‌های تحت مالکیت خود، لیست‌های فروش فعال و تاریخچه تراکنش‌هایشان را مشاهده و مدیریت کنند.

- **جزئیات توکن (/nftdetails/[tokenId]):** این صفحه به صورت یک کامپوننت سرور پیاده‌سازی شده است که اطلاعات توکن را مستقیماً از پایگاه داده دریافت می‌کند. همچنین برای نمایش قیمت دلاری، به صورت آنی قیمت اتریوم را از یک API خارجی استعلام می‌کند. کامپوننت‌های داخلی‌اش، NFTDescription و NFTDetails مسئولیت نمایش اطلاعات توکن، مدیریت عملیات خرید، افزودن به لیست علاقه‌مندی‌ها و گزارش تخلف را بر عهده دارند.

۵-۳-۲ مدیریت وضعیت و تعامل با بلاکچین

• Provider

برای مدیریت وضعیت سراسری و فراهم کردن قابلیت‌های کلیدی مانند اتصال به کیف پول، احراز هویت و تعامل با بلاکچین در سراسر برنامه، یک کامپوننت مرکزی به نام Providers ایجاد شده است. این کامپوننت، کل برنامه را در بر می‌گیرد و تضمین می‌کند که تمام صفحات و کامپوننت‌های فرزند به Context ها و هوک‌های مورد نیاز دسترسی داشته باشند.

این کامپوننت از چهار Provider اصلی تشکیل شده است:

1. **WagmiProvider:** این Provider هسته اصلی تعامل با بلاکچین است. با استفاده از getDefaultConfig، پیکربندی لازم برای اتصال به شبکه‌های اتریوم (شامل شبکه اصلی mainnet و شبکه محلی hardhatLocal) انجام می‌شود. این Provider به تمام کامپوننت‌های برنامه اجازه می‌دهد تا از هوک‌های wagmi برای خواندن داده‌ها از بلاکچین و ارسال تراکنش‌ها استفاده کنند.
2. **RainbowKitProvider:** این Provider مسئولیت نمایش دکمه "Connect Wallet"، پنجره انتخاب کیف پول، نمایش اطلاعات حساب کاربری و مدیریت فرآیند تغییر شبکه را بر عهده دارد.
3. **SessionProvider:** این Provider از کتابخانه NextAuth.js می‌آید و وظیفه مدیریت جلسات احراز هویت کاربر را دارد. پس از اینکه کاربر با موفقیت از طریق فرآیند SIWE وارد سیستم شد، اطلاعات جلسه او در این Provider نگهداری می‌شود و از طریق هوک useSession در دسترس قرار می‌گیرد.
4. **QueryClientProvider:** این Provider از کتابخانه TanStack Query برای مدیریت وضعیت داده‌های سمت سرور استفاده می‌کند. این ابزار به بهینه‌سازی درخواست‌های API، کش کردن داده‌ها و جلوگیری از درخواست‌های تکراری کمک کرده و در نتیجه عملکرد و تجربه کاربری را بهبود می‌بخشد.

• Context

برای مدیریت ارتباط با بلاکچین، از یک React Context به نام NFTMarketplaceContext استفاده شده است. این فایل، توابع اصلی تعامل با قرارداد هوشمند را با استفاده از هوک‌های کتابخانه Wagmi پیاده‌سازی کرده و آن‌ها را در اختیار تمام اجزای رابط کاربری قرار می‌دهد. تمام فرآیندهای پیچیده مانند ساخت، خرید یا فروش مجدد توکن در این Context مدیریت می‌شوند.

در این Context از مجموعه‌ای از هوک‌های کتابخانه wagmi برای تعامل با بلاکچین استفاده شده است:

- **useAccount**: برای دسترسی به اطلاعات حساب کاربری متصل شده، مانند آدرس کیف پول.
- **usePublicClient**: برای ایجاد یک کلاینت امن و Read-only جهت فراخوانی توابعی از قرارداد هوشمند که نیازی به ارسال تراکنش ندارند (مانند دریافت اطلاعات).
- **useWriteContract**: برای ارسال تراکنش‌ها و اجرای توابعی که وضعیت بلاکچین را تغییر می‌دهند (مانند ساخت، خرید یا فروش توکن).

```
const publicClient = usePublicClient(); /* read-only EVM client for readContract
const { writeContractAsync } = useWriteContract(); /* writeContract
publicClient.readContract({
  address: NFTMarketplaceAddress,
  abi: NFTMarketplaceABI,
  functionName: "getUserBalanceOf",
  args: [address],
}); /* call on-chain function
writeContractAsync({
  address: NFTMarketplaceAddress,
  abi: NFTMarketplaceABI,
  functionName: "withdraw",
  args: [parseEther(String(amountEth))],
}); /* call on-chain function
```

شکل ۵-۲: نمونه کد استفاده از تابع قرارداد در فرانت‌اند

در زیر می‌توانید تعاملات و روش اجرای چند نمونه از توابع این بستر را ببینید:

- **createSale**: این تابع پیچیده‌ترین فرآیند را مدیریت می‌کند. ابتدا با فراخوانی تابع listingFeeFor از قرارداد هوشمند، کارمزد لیست کردن توکن را محاسبه می‌کند. سپس، تابع createToken را بر روی قرارداد هوشمند فراخوانی کرده و tokenURI، قیمت و کارمزد لیست کردن را به عنوان پارامتر ارسال می‌کند. پس از موفقیت‌آمیز بودن تراکنش، رویداد MarketItemCreated را از لاگ‌های تراکنش استخراج کرده تا tokenId جدید را به دست آورد. در نهایت، اطلاعات توکن را از طریق یک درخواست POST به مسیر (/api/create-nft/save-nft) ارسال می‌کند تا در پایگاه داده ذخیره شود.

- **fetchMyNFTs**: این تابع با فراخوانی متد fetchMyNFTs از قرارداد هوشمند، لیست توکن‌های تحت مالکیت کاربر فعلی را دریافت می‌کند. سپس با پیمایش این لیست، tokenURI هر توکن را استعلام کرده و با ارسال درخواست به IPFS از طریق کتابخانه axios، اطلاعات مربوط به هر توکن (شامل نام، توضیحات و آدرس تصویر) را دریافت و به فرانت‌اند باز می‌گرداند.

```

//$ fetch user NFTs
const fetchMyNFTs = async () => {
  try {
    /** validation
    if (!address) throw new Error("Please connect a wallet first.");

    const data = await publicClient.readContract({
      address: NFTMarketplaceAddress,
      abi: NFTMarketplaceABI,
      functionName: "fetchMyNFTs",
      account: address,
    }); /** on-chain call

    /** circulate on-chain data
    const items = await Promise.all(
      /** map fetched on-chain data
      data.map(async (item) => {
        const tokenId = Number(item.tokenId);
        const tokenURI = await publicClient.readContract({
          address: NFTMarketplaceAddress,
          abi: NFTMarketplaceABI,
          functionName: "tokenURI",
          args: [BigInt(tokenId)],
        }); /** get token pinata(data) url

        let meta = {};
        try {
          const res = await axios.get(tokenURI); /** get token data from pinata
          meta = res?.data || {};
        } catch (e) {
          console.warn("tokenURI fetch failed:", tokenURI, e);
        }

        return {
          tokenId,
          seller: item.seller,
          owner: item.owner,
          price: Number(formatEther(item.price)),
          image: meta.image,
          name: meta.name,
          description: meta.description,
          tokenURI,
        };
      });

    return items;
  } catch (error) {
    console.error("Error fetching NFTs:", error);
    return [];
  }
};

```

شکل ۵-۵: کد تابع fetchMyNFTs در فایل NFTMarketplaceContext

۴-۵ پیاده‌سازی احراز هویت

یکی از نیازمندی‌های کلیدی این پروژه، ایجاد یک سیستم ورود امن، مدرن و سازگار با اکوسیستم Web3 بود. به جای استفاده از روش‌های سنتی مبتنی بر نام کاربری و رمز عبور، از پروتکل SIWE استفاده شده است. فرآیند احراز هویت شامل تعامل هماهنگ بین فرانت‌اند، بک‌اند و کیف پول کاربر است که در ادامه جزئیات پیاده‌سازی هر بخش تشریح می‌شود.

۴-۵-۱ پیاده‌سازی سمت بک‌اند (NextAuth.js)

هسته اصلی منطق احراز هویت در بک‌اند و در مسیر src/app/api/auth/[...nextauth]/route.js پیاده‌سازی شده است. در این فایل، از کتابخانه NextAuth.js و یک CredentialsProvider اختصاصی برای مدیریت فرآیند SIWE استفاده شده است.

تابع authorize در این Provider، قلب این فرآیند است و مراحل زیر را طی می‌کند:

- **دریافت پیام و امضا**: پیام و امضای دیجیتال ارسال‌شده از سمت کلاینت را دریافت می‌کند.
- **ساخت شیء SIWE**: پیام دریافتی را که به صورت رشته JSON است، به یک شیء SiweMessage تبدیل می‌کند.

- **دریافت Nonce:** برای جلوگیری از حملات تکرار^۱، یک توکن CSRF که شامل یک Nonce^۲ است، از کوکی‌های درخواست استخراج می‌شود.
- **اعتبارسنجی امضا:** با استفاده از متد `siwe.verify()`، اعتبار امضای کاربر را بررسی می‌کند. این متد سه مورد کلیدی را تأیید می‌کند:
 - امضا با کلید خصوصی متناظر با آدرس عمومی موجود در پیام، تولید شده باشد.
 - دامنه وب‌سایت با آنچه در پیام ذکر شده، مطابقت داشته باشد.
 - مقدار nonce موجود در پیام با nonce ذخیره‌شده در جلسه سرور یکسان باشد.

```

async authorize(credentials) {
  try {
    const siwe = new SiweMessage(
      JSON.parse(credentials?.message || "{}")
    );
    const raw = cookies().get("next-auth.csrf-token")?.value || "";
    const nonce = raw.split("|")[0] || "";

    const result = await siwe.verify({
      signature: credentials?.signature || "",
      domain: new URL(process.env.NEXTAUTH_URL).host,
      nonce,
    });

    if (result.success) {
      const walletAddress = siwe.address.toLowerCase();
      const user = await prisma.user.upsert({
        where: { walletAddress },
        update: {},
        create: { walletAddress },
      });
      return { id: user.id, address: walletAddress };
    }
    return null;
  } catch (e) {
    return null;
  }
}

```

شکل ۵-۶: کد تابع `authorize` در API احراز هویت

- **مدیریت کاربر در پایگاه داده:** پس از تأیید موفقیت‌آمیز، آدرس کیف پول کاربر از پیام استخراج می‌شود. سپس با استفاده از تابع `prisma.user.upsert`، بررسی می‌شود که آیا کاربری با این آدرس در پایگاه داده وجود دارد یا خیر. اگر وجود نداشت، یک کاربر جدید ایجاد می‌شود؛ در غیر این صورت، اطلاعات کاربر موجود بازگردانده می‌شود.
- **ایجاد جلسه:** در نهایت، یک شیء کاربر شامل `id` و `address` به `NextAuth` بازگردانده می‌شود تا جلسه برای کاربر ایجاد شود.

۵-۴-۲ پیاده‌سازی سمت فرانت‌اند

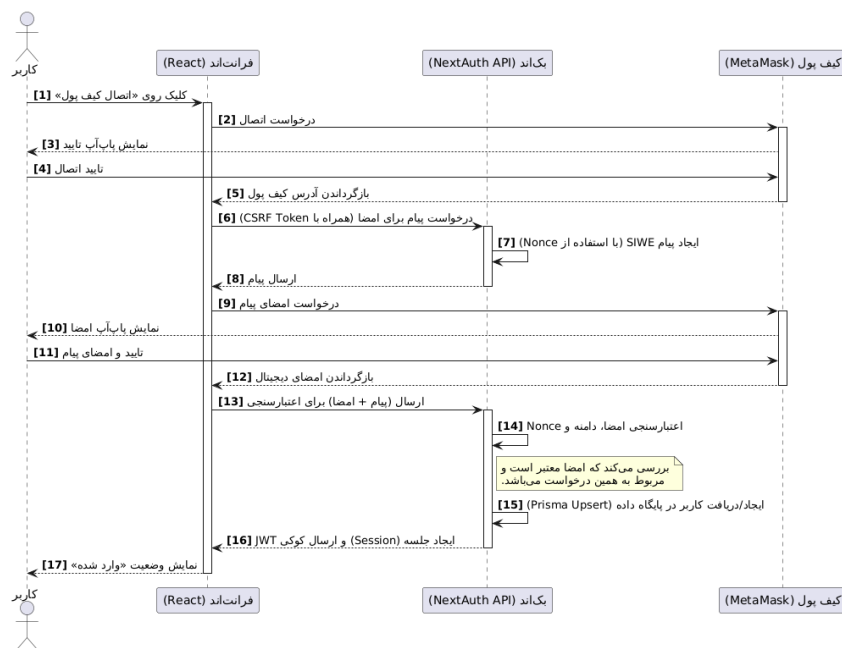
در سمت فرانت‌اند، کامپوننت `Header` به عنوان نقطه شروع فرآیند احراز هویت عمل می‌کند. با استفاده از `useEffect`، وضعیت اتصال کیف پول به طور مداوم بررسی می‌شود.

تابع `handleLogin` که در این کامپوننت تعریف شده، مسئول اجرای مراحل زیر در سمت کلاینت است:

^۱ Replay attacks

عدد تصادفی^۲

1. دریافت **CSRF Token**: ابتدا یک درخواست به مسیر `/api/auth/csrf` ارسال می‌شود تا یک توکن CSRF که حاوی nonce مورد نیاز برای پیام SIWE است، دریافت شود.
2. ایجاد پیام **SIWE**: با استفاده از کتابخانه `siwe`، یک پیام استاندارد ساخته می‌شود که شامل اطلاعاتی مانند دامنه، آدرس کاربر، شناسه زنجیره و nonce دریافت شده است.
3. درخواست امضا از کاربر: با استفاده از هوک `signMessageAsync` از کتابخانه `wagmi`، از کاربر خواسته می‌شود تا پیام ساخته شده را با کیف پول خود امضا کند. این عمل، پنجره‌ای را در `MetaMask` باز می‌کند که محتوای پیام را به کاربر نمایش می‌دهد.
4. ارسال اطلاعات به بک‌اند: پس از دریافت امضا، تابع `signIn` از `NextAuth.js` فراخوانی می‌شود. این تابع، پیام اصلی و امضای کاربر را به `CredentialsProvider` در بک‌اند ارسال می‌کند تا فرآیند اعتبارسنجی که در بخش قبل توضیح داده شد، انجام شود.



شکل ۵-۷: نمودار توالی برای فرآیند ورود و احراز هویت

فصل ششم

پیاده‌سازی و اجرا

این فصل به توضیح مراحل لازم برای راه‌اندازی، اجرا و تست کامل پروژه می‌پردازد. بخش اول، راهنمای مرحله به مرحله اجرای پروژه از راه پوشش می‌دهد و بخش دوم به روش‌های تست و اعتبارسنجی عملکرد سیستم می‌پردازد.

۶-۱ راهنمای اجرای پروژه

برای اجرای کامل این پلتفرم، لازم است که هر سه لایه اصلی بلاکچین، بک‌اند و فرانت‌اند به درستی پیکربندی و اجرا شوند. مراحل زیر این فرآیند را به ترتیب شرح می‌دهند.

۶-۱-۱ پیش‌نیازها و نصب وابستگی‌ها

اولین قدم، نصب تمام کتابخانه‌ها و بسته‌های نرم‌افزاری مورد نیاز پروژه است. این وابستگی‌ها در فایل `package.json` تعریف شده‌اند. برای نصب آن‌ها، کافی است دستور زیر در ریشه پوشه `site` اجرا شود:

`npm install`

۶-۱-۲ پیکربندی متغیرهای محیطی

برنامه برای عملکرد صحیح به چندین متغیر محیطی نیاز دارد که باید در یک فایل با نام `env` در ریشه پوشه `site` تعریف شوند. این متغیرها شامل کلیدهای API، آدرس‌ها و اطلاعات پیکربندی است.

```
1. # SQLite آدرس اتصال به پایگاه داده
2. DATABASE_URL="file:./dev.db"
3.
4. # NextAuth آدرس اصلی برنامه برای
5. NEXTAUTH_URL="http://localhost:3000"
6.
7. # NextAuth برای JWT یک کلید محرمانه برای رمزنگاری توکن های
8. NEXTAUTH_SECRET="secret"
9.
10. # آدرس قرارداد هوشمند پس از استقرار
11. NEXT_NFT_MARKETPLACE_ADDRESS="0x..."
12.
13. # Pinata کلیدهای سرویس
14. PINATA_API_KEY="YOUR_PINATA_API_KEY"
15. PINATA_SECRET_API_KEY="YOUR_PINATA_SECRET_KEY"
16.
17. # WalletConnect آیدی پروژه
18. NEXT_PUBLIC_WALLETCONNECT_PROJECT_ID="YOUR_WALLETCONNECT_PROJECT_ID"
19.
```

۶-۱-۳ راه‌اندازی و مدیریت پایگاه داده

پایگاه داده پروژه با استفاده از Prisma مدیریت می‌شود. برای ایجاد فایل پایگاه داده SQLite و ساخت جداول بر اساس شمای تعریف شده در `prisma/schema.prisma`، دستور زیر اجرا می‌شود:

`npx prisma migrate dev --name init`

این دستور، تغییرات مدل داده را اعمال کرده و پایگاه داده را برای استفاده آماده می‌کند. برای مشاهده و مدیریت بصری داده‌ها، می‌توان از ابزار Prisma Studio با دستور زیر استفاده کرد:

`npx prisma studio`

۶-۱-۴ اجرای شبکه بلاکچین محلی

برای تست و توسعه، یک شبکه بلاکچین محلی با استفاده از Hardhat راه‌اندازی می‌شود. این کار با دستور زیر انجام می‌شود:

`npx hardhat node`

پس از اجرای این دستور، یک شبکه محلی بر روی آدرس <http://127.0.0.1:8545> فعال شده و ۲۰ حساب کاربری به همراه کلیدهای خصوصی آن‌ها برای تست در اختیار قرار می‌گیرد. آیدی، ارز دیجیتال شبکه و آدرس شبکه در فایل Providers قابل تغییر هستند.

```
E:\DEV\NFTMarketplace\site>npx hardhat node
Started HTTP and WebSocket JSON-RPC server at http://127.0.0.1:8545/

Accounts
=====

WARNING: These accounts, and their private keys, are publicly known.
Any funds sent to them on Mainnet or any other live network WILL BE LOST.

Account #0: 0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266 (10000 ETH)
Private Key: 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80

Account #1: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 (10000 ETH)
Private Key: 0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d

Account #2: 0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC (10000 ETH)
Private Key: 0x5de4111afa1a4b94908f83103eb1f1706367c2e68ca870fc3fb9a804cdab365a

Account #3: 0x90f79bf6EB2c4f870365E785982E1f101E93b906 (10000 ETH)
Private Key: 0x7c852118294e51e653712a81e05800f419141751be58f605c371e15141b007a6

Account #4: 0x15d34AAf54267DB7D7c367839AAf71A00a2C6A65 (10000 ETH)
Private Key: 0x47e179ec197488593b187f80a00eb0da91f1b9d0b13f8733639f19c30a34926a
```

شکل ۶-۱: خروجی دستور اجرای hardhat

۶-۱-۵ کامپایل و استقرار قرارداد هوشمند

پس از اجرای شبکه محلی، قرارداد هوشمند باید کامپایل و بر روی آن مستقر شود. این کار با اجرای اسکریپت deploy.cjs از طریق دستور زیر انجام می‌شود:

```
npx hardhat run scripts/deploy.cjs --network localhost
```

خروجی این دستور، آدرس قرارداد هوشمند مستقر شده است. این آدرس باید کپی شده و به عنوان مقدار متغیر NEXT_NFT_MARKETPLACE_ADDRESS در فایل env قرار گیرد.

```
E:\DEV\NFTMarketplace\site>npx hardhat run scripts/deploy.cjs --network localhost
Compiled 24 Solidity files successfully (evm target: paris).
NFTMarketplace deployed to: 0x5FbDB2315678afecb367f032d93F642f64180aa3
```

```
hardhat_addCompilationResult
eth_accounts
hardhat_metadata (20)
eth_blockNumber
eth_getBlockByNumber
eth_feeHistory
eth_maxPriorityFeePerGas
eth_sendTransaction
Contract deployment: NFTMarketplace
Contract address: 0x5fbdb2315678afecb367f032d93f642f64180aa3
Transaction: 0x429465ece91414097658eec0b14f0c5c33a060b7d730b08b14a4cb01a39e690f
From: 0xf39fd6e51aad88f6f4ce6ab8827279cFfFb92266
Value: 0 ETH
Gas used: 4389493 of 30000000
Block #1: 0x8b7dcbbbc5edaa217d39ca452b8438ea218a6004bb933a49edf78dc53d40a9f1
```

شکل ۶-۲: خروجی دستور پیاده‌سازی قرارداد

۶-۱-۶ انتقال ABI

پس از کامپایل و استقرار قرارداد هوشمند ، یک فایل JSON به نام NFTMarketplace.json در مسیر site/artifacts/contracts/SmartContract.sol/ ایجاد می‌شود. این فایل شامل ABI^۱ قرارداد است که نحوه تعامل با توابع آن را تعریف می‌کند.

برای اینکه فرانت‌اند بتواند با قرارداد هوشمند ارتباط برقرار کند، محتوای این فایل JSON باید به فایل src/context/NFTMarketplace.json منتقل شود.

۶-۱-۷ اجرای سرور توسعه

در نهایت، با اجرای دستور زیر، سرور توسعه Next.js فعال شده و برنامه بر روی آدرس <http://localhost:3000> قابل دسترسی خواهد بود:

npm run dev

پس از انجام این مراحل، پلتفرم به طور کامل بر روی محیط محلی اجرا شده و آماده تعامل از طریق کیف پول MetaMask است.

۶-۱-۸ پیکربندی کیف پول MetaMask برای تست

برای تعامل با پلتفرم در شبکه محلی، لازم است کیف پول مرورگر MetaMask به درستی پیکربندی شود. این فرآیند شامل دو مرحله اصلی است: افزودن شبکه محلی Hardhat و وارد کردن حساب‌های کاربری تست.

• افزودن شبکه محلی Hardhat

برای اینکه MetaMask بتواند با بلاکچین محلی که توسط Hardhat اجرا شده ارتباط برقرار کند، باید اطلاعات این شبکه به صورت دستی به آن اضافه شود.

۱. افزونه MetaMask را در مرورگر خود باز کنید.

۲. بر روی منوی انتخاب شبکه کلیک کرده و گزینه «Add network» را انتخاب کنید.

۳. در صفحه‌ای که باز می‌شود، گزینه «Add a custom network» را انتخاب کنید.

¹ Application binary interface

۴. فرم را با اطلاعات زیر پر کنید. این اطلاعات مطابق با پیکربندی شبکه در فایل `hardhat.config.cjs` و خروجی دستور `npx hardhat node` است:

Network Name: Hardhat Localhost

New RPC URL: `http://127.0.0.1:8545`

Chain ID: 1337

Currency Symbol: ETH

۵. پس از ذخیره کردن، شبکه Hardhat Localhost به لیست شبکه‌های شما اضافه شده و می‌توانید آن را انتخاب کنید.

شکل ۶-۳: صفحه تنظیمات شبکه محلی در metamask

• وارد کردن حساب‌های کاربری تست

پس از اجرای دستور `npx hardhat node`، مجموعه‌ای از حساب‌های کاربری به همراه کلیدهای خصوصی آن‌ها در ترمینال نمایش داده می‌شود. برای شبیه‌سازی تعامل کاربران مختلف، باید این حساب‌ها را به MetaMask وارد کرد.

۱. در MetaMask، روی آیکون پروفایل خود کلیک کرده و گزینه «Import account» را انتخاب کنید.

۲. یکی از کلیدهای خصوصی نمایش داده شده در خروجی Hardhat را کپی کنید. برای آسانی، چهار عدد از این کلیدها در صفحه اصلی پروژه نیز نمایش داده شده‌اند.

۳. کلید خصوصی را در فیلد مربوطه وارد کرده و روی دکمه «Import» کلیک کنید.

۴. حساب کاربری جدید با موجودی ۱۰۰۰۰ اتر مجازی به لیست حساب‌های شما در MetaMask اضافه می‌شود.

۲-۶ راهنمای اجرای پروژه

پس از اجرای موفقیت‌آمیز پروژه، برای اطمینان از عملکرد صحیح تمام قابلیت‌ها، مجموعه‌ای از سناریوهای تست دستی طراحی و اجرا شد. این تست‌ها با استفاده از حساب‌های کاربری مختلف که از طریق کلیدهای خصوصی Hardhat به MetaMask اضافه شده‌اند، انجام گرفت تا تعاملات بین کاربران مختلف شبیه‌سازی شود. در ادامه، مراحل و نتایج مورد انتظار برای هر یک از این سناریوها تشریح می‌شود.

۱-۲-۶ تست ورود و امضای دیجیتال

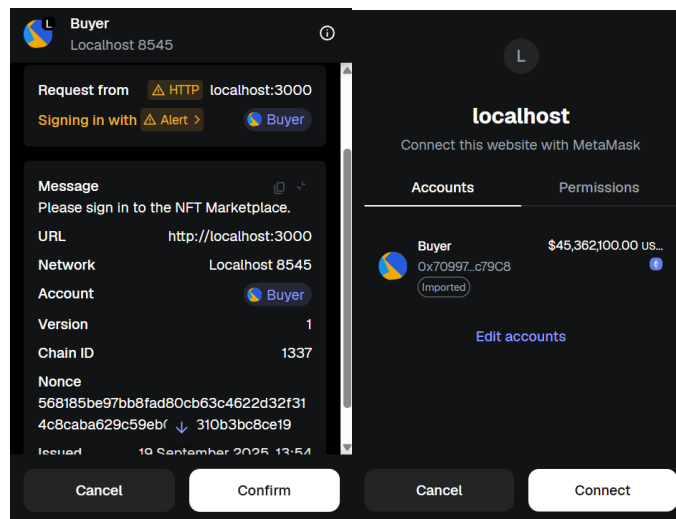
• **هدف:** اعتبارسنجی فرآیند کامل احراز هویت کاربر از طریق کیف پول.

• **مراحل تست:**

1. کاربر بر روی دکمه «Connect Wallet» در سربرگ سایت کلیک می‌کند.
2. در پنجره باز شده از RainbowKit، کیف پول MetaMask را انتخاب می‌کند.
3. MetaMask برای اتصال حساب کاربری درخواست مجوز می‌دهد؛ کاربر آن را تأیید می‌کند.
4. بلافاصله پس از اتصال، MetaMask یک پنجره جدید برای امضای یک پیام دیجیتال نمایش می‌دهد. کاربر پیام را امضا می‌کند.

• **نتیجه مورد انتظار:**

- پس از امضای موفق، کاربر به عنوان احراز هویت شده شناخته می‌شود.
- دکمه «Connect Wallet» با آدرس کیف پول کاربر جایگزین می‌شود.
- لینک «Profile» در سربرگ نمایان می‌شود.
- یک رکورد جدید برای کاربر جدید در جدول User پایگاه داده ایجاد می‌شود.



شکل ۶-۴: صفحه ورود و امضا در metamask

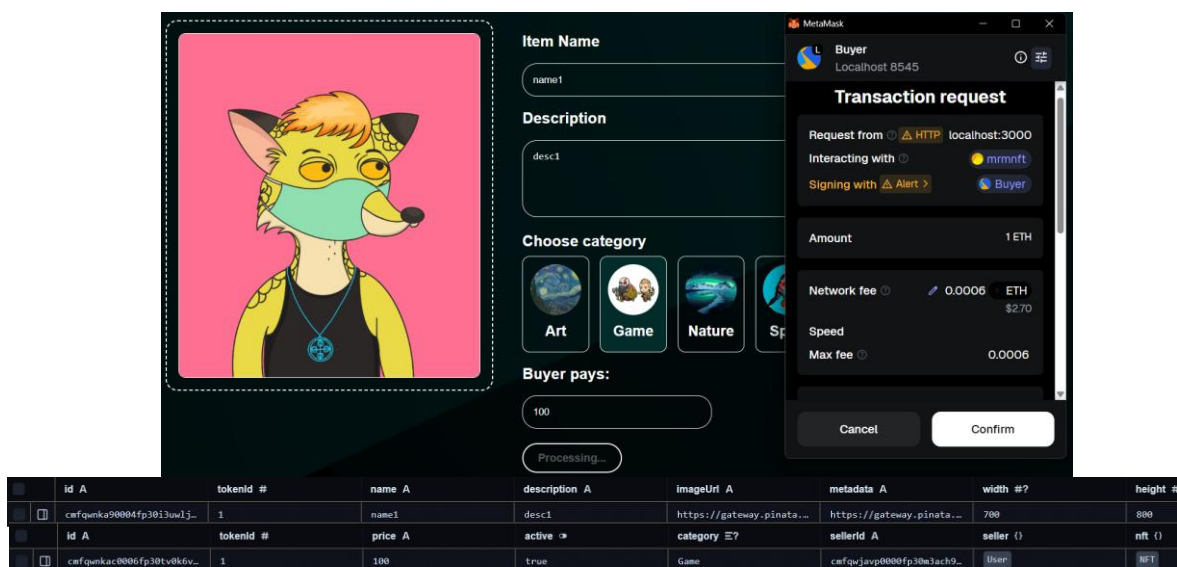
۶-۲-۲ تست ساخت و لیست کردن توکن

- هدف: اطمینان از عملکرد صحیح فرآیند کامل ساخت، آپلود قرارداد و لیست کردن یک توکن برای فروش.
- مراحل تست:

1. کاربر وارد شده به صفحه «Create NFT» مراجعه می کند.
2. یک فایل تصویر را در کامپوننت DropZone بارگذاری می کند.
3. فیلدهای نام، توضیحات، قیمت و دسته بندی را تکمیل می کند.
4. بر روی دکمه «Create and list NFT» کلیک می کند.
5. MetaMask یک تراکنش را برای تأیید نمایش می دهد که شامل کارمزد ۱٪ برای لیست کردن است. کاربر تراکنش را تأیید می کند.

• نتیجه مورد انتظار:

- تراکنش با موفقیت در بلاکچین محلی ثبت می شود.
- توکن جدید در صفحه «Marketplace» و همچنین در تب «Active Listings» پروفایل کاربر قابل مشاهده است.
- مالکیت توکن در قرارداد هوشمند به آدرس خود قرارداد منتقل می شود.
- رکوردهای جدیدی در جداول NFT و Listing در پایگاه داده ایجاد می شود.



شکل ۶-۵: صفحه ساخت توکن و نتیجه در پایگاه داده

۳-۲-۶ تست خرید یک توکن

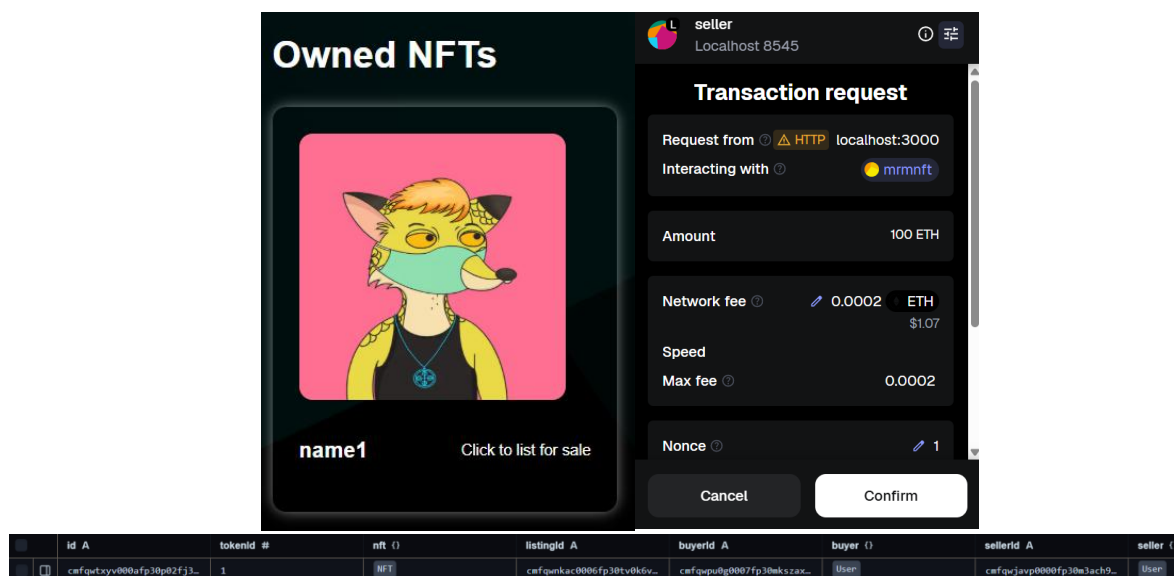
- هدف: اعتبارسنجی فرآیند خرید یک توکن توسط یک کاربر دیگر.

• مراحل تست:

1. کاربر دوم (خریدار) با یک حساب کاربری متفاوت وارد سیستم می‌شود.
2. به صفحه بازار یا صفحه جزئیات توکن ساخته شده در مرحله قبل مراجعه می‌کند.
3. بر روی دکمه «Buy NFT» کلیک می‌کند.
4. MetaMask یک تراکنش به ارزش قیمت کامل توکن نمایش می‌دهد. خریدار تراکنش را تأیید می‌کند.

• نتیجه مورد انتظار:

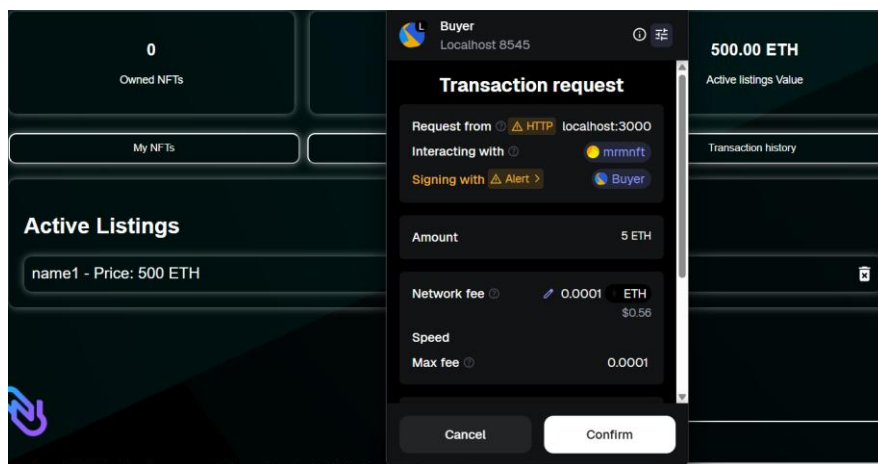
- تراکنش با موفقیت ثبت می‌شود.
- مالکیت توکن در قرارداد هوشمند از آدرس قرارداد به آدرس کیف پول خریدار منتقل می‌شود.
- توکن از صفحه بازار حذف می‌شود.
- توکن در تب «My NFTs» پروفایل خریدار نمایش داده می‌شود.
- مبلغ فروش (پس از کسر کارمزد) به موجودی قابل برداشت فروشنده در balances قرارداد هوشمند اضافه می‌شود.
- یک رکورد جدید در جدول Transaction پایگاه داده ثبت می‌شود و رکورد Listing مربوطه غیرفعال (active: false) می‌شود.



شکل ۶-۶: صفحه خرید توکن و نتیجه در جدول معاملات پایگاه داده و پروفایل

۶-۲-۴ تست لغو فروش

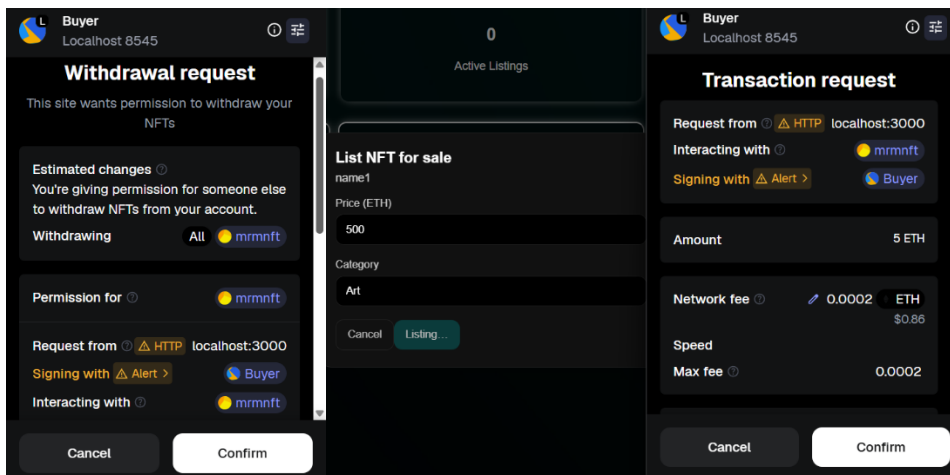
- **هدف:** اطمینان از اینکه فروشنده می‌تواند فروش یک توکن را لغو کند.
- **مراحل تست:**
 1. کاربر فروشنده وارد پروفایل خود شده و به تب «Active Listings» مراجعه می‌کند.
 2. بر روی آیکون لغو در کنار توکن مورد نظر کلیک می‌کند.
 3. MetaMask تراکنشی برای پرداخت کارمزد ۱٪ لغو فروش نمایش می‌دهد. کاربر آن را تأیید می‌کند.
- **نتیجه مورد انتظار:**
 - تراکنش با موفقیت ثبت می‌شود.
 - توکن از لیست فروش‌های فعال کاربر حذف شده و به تب «My NFTs» او منتقل می‌شود.
 - مالکیت توکن در قرارداد هوشمند از آدرس قرارداد به آدرس فروشنده بازگردانده می‌شود.
 - رکورد Listing در پایگاه داده غیرفعال می‌شود.



شکل ۶-۷: صفحه کنسل کردن فروش

۵-۲-۶ تست فروش مجدد

- **هدف:** اعتبارسنجی فرآیند لیست کردن مجدد یک توکن که در مالکیت کاربر قرار دارد.
- **مراحل تست:**
 1. کاربر مالک یک توکن (که در حال فروش نیست) وارد پروفایل خود شده و به تب «MyNFTs» مراجعه می‌کند.
 2. بر روی توکن مورد نظر کلیک می‌کند.
 3. در پنجره باز شده، قیمت و دسته‌بندی جدید را وارد کرده و دکمه «List» را می‌زند.
 4. MetaMask ابتدا (در صورت نیاز) یک تراکنش برای تأیید اجازه دسترسی (Approval) و سپس تراکنش اصلی برای لیست کردن (شامل کارمزد ۱٪) را نمایش می‌دهد. کاربر تراکنش‌ها را تأیید می‌کند.
- **نتیجه مورد انتظار:**
 - تراکنش با موفقیت ثبت می‌شود.
 - توکن از تب «My NFTs» حذف شده و به تب «Active Listings» منتقل می‌شود.
 - مالکیت توکن مجدداً در قرارداد هوشمند به آدرس خود قرارداد منتقل می‌شود.
 - رکورد Listing در پایگاه داده با اطلاعات جدید به‌روزرسانی می‌شود.



شکل ۶-۸: صفحه فروش مجدد توکن و اجازه برداشت توکن

۶-۲-۶ تست لیست علاقه‌مندی‌ها و گزارش تخلف

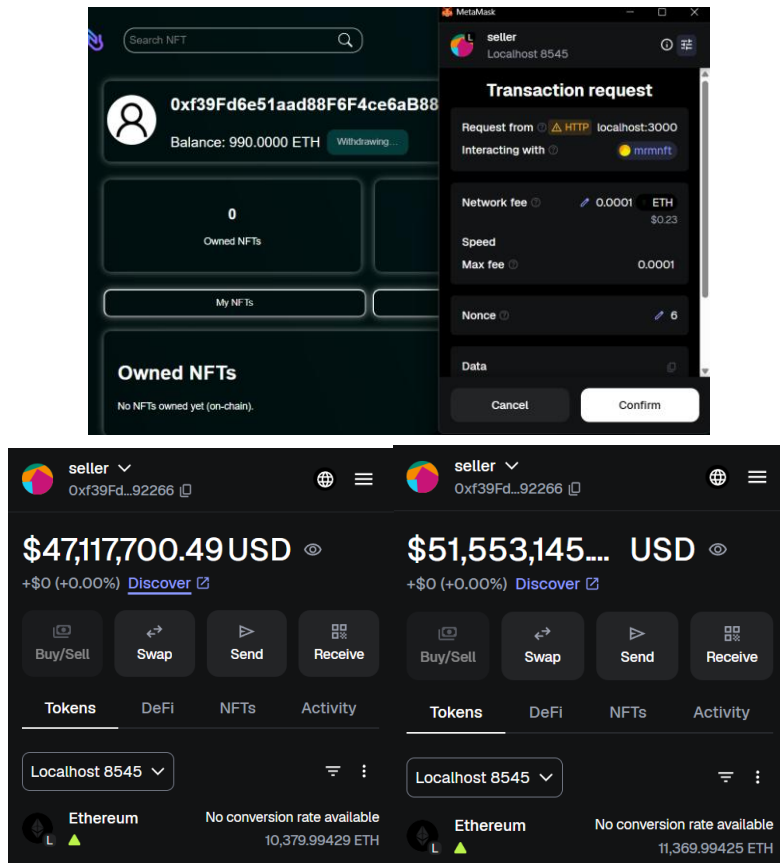
- **هدف:** بررسی عملکرد صحیح قابلیت‌های جانبی مانند افزودن به لیست علاقه‌مندی‌ها و گزارش تخلف.
- **مراحل تست:**
 1. کاربر به صفحه جزئیات یک توکن مراجعه می‌کند.
 2. از طریق منوی سه نقطه، گزینه «Add to WishList» را انتخاب می‌کند. سپس با کلیک بر روی آیکون Wishlist در Header، به صفحه بازار با فیلتر wishlist مراجعه می‌کند.
 3. در صفحه جزئیات گزینه «Report» را انتخاب می‌کند.
- **نتیجه مورد انتظار:**
 - پس از افزودن به علاقه‌مندی‌ها، توکن در صفحه فیلتر شده نمایش داده می‌شود.
 - پس از گزارش، یک رکورد جدید در جدول Report پایگاه داده ثبت شده و پیام موفقیت به کاربر نمایش داده می‌شود.

۶-۲-۷ تست برداشت وجه

- **هدف:** اطمینان از اینکه فروشنده می‌تواند درآمد حاصل از فروش خود را برداشت کند.
- **مراحل تست:**
 1. کاربر فروشنده (که قبلاً توکنی را فروخته است) وارد صفحه پروفایل خود می‌شود.
 2. موجودی قابل برداشت او در بالای صفحه نمایش داده می‌شود.
 3. بر روی دکمه «Withdraw» کلیک می‌کند.
 4. MetaMask یک تراکنش برای تأیید نمایش می‌دهد. کاربر آن را تأیید می‌کند.

نتیجه مورد انتظار:

- تراکنش با موفقیت ثبت می‌شود.
- مبلغ موجودی از قرارداد هوشمند به کیف پول فروشنده منتقل می‌شود.
- موجودی قابل برداشت او در پروفایل و در قرارداد هوشمند صفر می‌شود.



شکل ۶-۹: صفحه برداشت موجودی و تغییرات کیف پول

۶-۲-۸ تست قابلیت توقف اضطراری قرارداد

- **هدف:** اعتبارسنجی مکانیزم امنیتی توقف اضطراری قرارداد. این تست بررسی می‌کند که:
 1. مالک قرارداد می‌تواند توابع `pause` و `unpause` را فراخوانی کند.
 2. در حالت متوقف، توابع حساس و اصلی بازار از کار می‌افتند.
 3. پس از فعال‌سازی مجدد، عملکرد عادی قرارداد باز می‌گردد.
- **روش تست:** برای این تست، از کنسول تعاملی Hardhat استفاده می‌شود که امکان فراخوانی مستقیم توابع قرارداد هوشمند را فراهم می‌کند.
- **مراحل تست:**

1. ابتدا با دستور زیر، کنسول Hardhat در شبکه محلی اجرا می‌شود:

فصل هفتم

نتیجه‌گیری و کارهای آینده

۱-۷ نتیجه‌گیری

پروژه با موفقیت به هدف اصلی خود، یعنی طراحی و پیاده‌سازی یک بازار آنلاین غیرمتمرکز برای توکن‌های غیرقابل معاوضه، دست یافت. این پلتفرم یک راهکار جامع و کاربردی است که به کاربران امکان می‌دهد با استفاده از فناوری‌های Web3، دارایی‌های دیجیتال خود را به سادگی مدیریت کنند. دستاوردهای کلیدی پروژه عبارتند از:

- **پیاده‌سازی کامل قابلیت‌های اصلی بازار:** تمامی عملکردهای اساسی یک بازار توکن غیرقابل معاوضه، شامل ساخت، لیست کردن برای فروش، خرید، فروش مجدد و لغو فروش، با موفقیت پیاده‌سازی شد. این منطق توسط یک قرارداد هوشمند امن و کارآمد بر روی بلاکچین مدیریت می‌شود.
- **ایجاد یک پلتفرم امن با احراز هویت مدرن:** با بهره‌گیری از پروتکل SIWE و کتابخانه NextAuth.js، یک سیستم احراز هویت امن، غیرمتمرکز و بدون نیاز به رمز عبور پیاده‌سازی شد. این روش، استاندارد طلایی برای ورود به برنامه‌های Web3 محسوب می‌شود و تجربه کاربری یکپارچه‌ای را فراهم می‌کند.

- **تضمین غیرمتمرکز بودن با IPFS:** با ذخیره‌سازی متادیتا و فایل‌های اصلی توکن‌ها بر روی IPFS، وابستگی به یک سرور مرکزی برای نگهداری دارایی‌های دیجیتال حذف شد. این موضوع پایداری و ماندگاری داده‌ها را تضمین می‌کند.

- **ارائه رابط کاربری ساده و ریسپانسیو:** فرانت‌اند پروژه با استفاده از React و Next.js به گونه‌ای طراحی شد که تجربه‌ای روان و جذاب را برای کاربران، حتی کسانی که آشنایی کمی با فناوری بلاکچین دارند، فراهم آورد. همچنین ساختار فرانت‌اند سیستم، به شکل ریسپانسیو طراحی شده تا برای کاربران دستگاه‌های مختلف، تجربه خوبی به ارمغان بیاورند.

در مجموع، این پروژه توانست با ترکیب فناوری‌های روی زنجیره مانند قراردادهای هوشمند و فناوری‌های خارج از زنجیره مانند Next.js، Prisma و IPFS، یک نمونه کامل و عملی از یک برنامه غیرمتمرکز را ارائه دهد که اهداف تعریف‌شده در ابتدای مسیر را به طور کامل پوشش می‌دهد.

۲-۷ کارهای آینده

برای توسعه و بهبود پلتفرم در آینده، می‌توان قابلیت‌های جدید و پیشرفته‌تری را به آن اضافه کرد. در ادامه، چند پیشنهاد کلیدی برای کارهای آینده ارائه می‌شود:

- **پشتیبانی از سیستم حراجی:** در حال حاضر، فروش توکن‌ها تنها با قیمت ثابت انجام می‌شود. افزودن یک مکانیزم حراجی می‌تواند یک روش جایگزین و جذاب برای فروش توکن‌ها فراهم کند. این قابلیت نیازمند توسعه منطق جدیدی در قرارداد هوشمند برای مدیریت پیشنهادات، زمان‌بندی و نهایی کردن حراج است و همچنین باید ساختار پایگاه‌داده، فرانت‌اند و بک‌اند تغییر کنند.
- **پشتیبانی از شبکه‌های بلاکچین دیگر:** این پروژه در حال حاضر بر روی یک شبکه سازگار با ماشین مجازی اتریوم اجرا می‌شود. با گسترش پشتیبانی به شبکه‌های دیگر مانند Polygon، Tron یا سایر بلاکچین‌های محبوب، که از برنامه‌های محیط ماشین مجازی اتریوم پشتیبانی می‌کنند، می‌توان هزینه‌های تراکنش را برای کاربران کاهش داد.
- **جداسازی قرارداد فروشگاه و ساخت توکن:** در معماری فعلی، منطق ساخت توکن و فروش آن در یک قرارداد هوشمند واحد قرار دارد. در نسخه‌های آینده، می‌توان این دو بخش را از هم جدا کرد. با این کار، یک قرارداد به عنوان کارخانه برای ساخت مجموعه‌های مختلف عمل کرده و قرارداد بازار به یک پلتفرم عمومی برای معامله هر نوع توکن استاندارد (ERC-721) تبدیل می‌شود. این معماری، انعطاف‌پذیری و قابلیت توسعه‌پذیری سیستم را به شدت افزایش می‌دهد.

- **افزودن دکمه توقف قرارداد در فرانت‌اند:** قابلیت توقف اضطراری قرارداد در حال حاضر در قرارداد هوشمند پیاده‌سازی شده است، اما تنها از طریق ابزارهای توسعه‌دهنده مانند Hardhat Console قابل فراخوانی است. می‌توان یک بخش مدیریتی در رابط کاربری ایجاد کرد که تنها برای مالک قرارداد قابل مشاهده باشد و به او اجازه دهد با یک کلیک، وضعیت توقف قرارداد را فعال یا غیرفعال کند. این ویژگی مدیریت پلتفرم در شرایط اضطراری را بسیار ساده‌تر و سریع‌تر می‌کند.

مراجع

- [1] www.udemy.com/course/the-complete-web-development-bootcamp/
- [2] udemy.com/course/blockchain-developer/
- [3] www.youtube.com/playlist?list=PLWUCKsxdKl0olgEF4OxXVk2B-jwpGqL5d
- [4] www.youtube.com/watch?v=y6JfVdcJh1k&t=7833s
- [5] www.youtube.com/watch?v=2bjVWclBD_s&t=6557s&pp=ygUPbmZ0IG1hcmtldHBsYWNI
- [6] github.com/alchemyplatform/NFT-Marketplace-Tutorial